

scenario_A_reproduce_v3

June 5, 2026

change everything to torch float64

```
[1]: # unify the data type into torch.float64
import torch
torch.set_default_dtype(torch.float64)
print("default dtype:", torch.get_default_dtype())

# cease warnings
import warnings

warnings.filterwarnings(
    "ignore",
    message=".*'where' used without 'out'.*",
    category=UserWarning,
    module="scipy.optimize._trustregion_constr"
)

warnings.filterwarnings(
    "ignore",
    message=".*delta_grad == 0.0.*",
    category=UserWarning,
    module="scipy.optimize._differentiable_functions"
)
```

default dtype: torch.float64

```
[2]: %load_ext autotime

# 0. Imports & path setup
import sys, os, copy, time, warnings
# sys.path.insert(0, "/home/niels/FELsim_clone/backend")
CURRENT_DIR = os.getcwd()
BACKEND_DIR = CURRENT_DIR if os.path.basename(CURRENT_DIR) == "backend" else os.
    ↪path.abspath(os.path.join(CURRENT_DIR, "../backend"))
sys.path.insert(0, BACKEND_DIR)
print(BACKEND_DIR)
os.makedirs(os.path.join(CURRENT_DIR, "../results"), exist_ok=True)
```

```

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("module://matplotlib_inline.backend_inline")
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.lines import Line2D

from ebeam import beam as ebeam_class
from beamline import lattice
from excelElements import ExcelElements
from beamOptimizer import beamOptimizer
from evolutionPlotter import EvolutionPlotter
from felsimAdapter import FELsimAdapter

# Shared constants
# EXCEL_PATH = "/home/niels/FELsim_clone/beam_excel/Beamline_elements_3.
# ↪xlsx"
# EXCEL_PATH = "/home/niels/FELsim_clone/beam_excel/Beamline_elements_3.
# ↪xlsx"
EXCEL_PATH = os.path.abspath(os.path.join(CURRENT_DIR, "../beam_excel/
↪Beamline_elements_3.xlsx"))

BEAM_ENERGY = 40.0 # MeV as the energy for a single electron
N_PARTICLES = 1000 # number of electrons in the beam for simulation
SEED = 42 # random seed for reproducibility
CURRENT_BOUNDS = (0.01, 1.5) # current bound in the experiment heuristically

# Relativistic factors
relat = lattice(1, fringeType=None)
relat.setE(E=BEAM_ENERGY) # set the energy of the relativistic factor
GAMMA_REL = relat.gamma # Lorentz factor as the mass ratio of the
↪electron and the static electron
BETA_REL = relat.beta # standardized electron velocity
NORM = GAMMA_REL * BETA_REL # electron momentum ratio between the current
↪electron and the static one

# Undulator matching targets (MkV FEL, K=1.2, λ_u=2.3 cm)
K = 1.2 # Undulator Parameter
LAMBDA_U = 2.3e-2 # wavelength, unit: m
## twiss parameters
BETA_YM = GAMMA_REL / (K * 2 * np.pi / LAMBDA_U) # matched λ_y at UND
↪entrance, beam size in y direction
BETA_XM = 1.4 # m - matched λ_x at UND entrance, beam size at the x
↪direction

```

```

ALPHA_XM = 0.47      # - Courant-Snyder _x at UND entrance, beam size converge
    ↳ at the direction x
ALPHA_YM = 0.0       # - Courant-Snyder _y at UND entrance, beam waist is
    ↳ achieved at the y direction

print(f" = {GAMMA_REL:.4f},      = {BETA_REL:.6f},      = {NORM:.4f}")
print(f"Matched _ym = {BETA_YM:.4f} m    (K={K}, _u={LAMBDA_U*100:.1f} cm)")
print(f"Matched _xm = {BETA_XM:.4f} m,    _xm = {ALPHA_XM:.4f},    _ym =
    ↳ {ALPHA_YM:.4f}")

```

```

c:\Users\yilu2\my_files\research\FELsim\backend
    = 79.2794,      = 0.999920,      = 79.2731
Matched _ym = 0.2418 m    (K=1.2, _u=2.3 cm)
Matched _xm = 1.4000 m,    _xm = 0.4700,    _ym = 0.0000
time: 4.92 s (started: 2026-06-05 15:05:14 -10:00)

```

0.1 1 — Beam Distribution

Physics-based 6D Gaussian matching the UH linac reference parameters.

```

[3]: %load_ext autotime

# Beam parameters
epsilon_n      = 8.0      # .mm.mrad normalised emittance, inner attribute
    ↳ of electron gun
x_std          = 0.8      # mm, initial standard deviation at x direction
y_std          = 0.8      # mm, initial standard deviation at y direction
f_RF           = 2856e6   # Hz, acceleration frequency
bunch_spread_ps = 2.0     # ps, beam length
energy_spread_pct = 0.5   # %, energy standard deviation inside the beam
h              = 5e9      # 1/s chirp, longitudinal chirp for beam
    ↳ compression

epsilon        = epsilon_n / NORM # geometric emittance
x_prime_std    = epsilon / x_std # alpha_x=0 at the initial point, so that sigma_x
    ↳ * sigma_x'=epsilon
y_prime_std    = epsilon / y_std # alpha_y=0 at the initial point, so that sigma_y
    ↳ * sigma_y'=epsilon
tof_std        = bunch_spread_ps * 1e-9 * f_RF # Time of Flight Standard
    ↳ Deviation, change from cycle to mili-cycle, measuring how much the electron
    ↳ bunch occupies the cycle
energy_std     = energy_spread_pct * 10 # transform from percent to per mill

ebeam_obj = ebeam_class()
dtype=torch.float64
PARTICLES = ebeam_obj.gen_6d_gaussian(
    0,

```

```

    [x_std, x_prime_std, y_std, y_prime_std, tof_std, energy_std],
    N_PARTICLES,
    dtype=dtype
)
# Apply longitudinal chirp
PARTICLES[:, 5] += h * (PARTICLES[:, 4] / f_RF)

print(f" = {epsilon:.5f} ·mm·mrad (geometric)")
print(f"_x={x_std:.2f} mm, _x'={x_prime_std:.5f} mrad")
print(f"_y={y_std:.2f} mm, _y'={y_prime_std:.5f} mrad")
print(f"Particles: {PARTICLES.shape}")

```

The autotime extension is already loaded. To reload it, use:

```

%reload_ext autotime
= 0.10092 ·mm·mrad (geometric)
_x=0.80 mm, _x'=0.12615 mrad
_y=0.80 mm, _y'=0.12615 mrad
Particles: torch.Size([1000, 6])
time: 16 ms (started: 2026-06-05 15:05:19 -10:00)

```

```

[4]: PARTICLES = PARTICLES.to(torch.float64)
print("PARTICLES dtype:", PARTICLES.dtype)

```

```

PARTICLES dtype: torch.float64
time: 0 ns (started: 2026-06-05 15:05:19 -10:00)

```

0.2 2 — Beamline & Element Index Map

The ExcelElements parser produces **138 FELsim elements**. Key reference indices: | Location | FELsim idx | z (m) | Physical meaning | |—|—|—|—| | End of 1st doublet region | 8–9 | 0.73–1.0 | First targets | | IP (focus) | 59 | 7.11 | Minimum envelope | | UND entrance | 117 | 12.39 | Undulator matching |

```

[5]: %load_ext autotime

# Load beamline & map quad indices
beamline_full = ExcelElements(EXCEL_PATH).create_beamline()
print(f"FELsim elements: {len(beamline_full)}")

s = 0.0
quad_info = []
for i, e in enumerate(beamline_full):
    cls = e.__class__.__name__
    L = float(getattr(e, 'length', 0.0) or 0.0)
    if cls in ('qpflattice', 'qpdLattice'):
        quad_info.append((i, cls, round(s, 5), round(s+L, 5),
                          round(float(getattr(e, 'current', 0) or 0), 5)))
    s += L

```

```

quad_indices = [q[0] for q in quad_info]
df_quads = pd.DataFrame(quad_info,
    columns=['idx', 'type', 's_start', 's_end', 'current_A'])
print(f"Quadrupoles: {len(quad_indices)}")
print(df_quads.to_string(index=False))

# Gold-standard currents from Eremey's FELsim S1 solution
FELSIM_S1_CURRENTS = {
    1: 0.8218, 3: 1.0430,
    10: 3.8834,
    16: 2.2396, 18: 4.9532, 20: 3.4258,
    27: 4.6657,
    33: 2.6942, 35: 2.6523, 37: 0.2768, 39: 0.2768, 41: 2.6523, 43: 2.6942,
    50: 4.6739,
    56: 3.1219, 58: 3.3129,
    61: 5.1775, 63: 4.0434,
    70: 4.6818,
    76: 3.9336, 78: 4.0787, 80: 0.0139,
    87: 1.3624, 93: 0.9452, 95: 2.8851, 97: 2.1921,
}
print(f"\nFELSIM_S1_CURRENTS: {len(FELSIM_S1_CURRENTS)} quadrupoles loaded")

```

The autotime extension is already loaded. To reload it, use:

```
%reload_ext autotime
```

FELsim elements: 138

Quadrupoles: 26

idx	type	s_start	s_end	current_A
1	qpfLattice	0.35878	0.44768	0.88572
3	qpdLattice	0.64453	0.73342	1.05289
10	qpfLattice	2.03304	2.12194	3.81660
16	qpdLattice	2.73535	2.82425	2.57981
18	qpfLattice	2.86743	2.95633	4.56570
20	qpdLattice	2.99951	3.08841	2.57981
27	qpfLattice	3.55448	3.64338	4.49146
33	qpdLattice	4.15187	4.24077	1.59614
35	qpfLattice	4.29157	4.38047	3.08092
37	qpdLattice	4.43127	4.52017	1.59614
39	qpdLattice	4.93513	5.02403	1.59614
41	qpfLattice	5.07483	5.16373	3.08092
43	qpdLattice	5.21453	5.30343	1.59614
50	qpfLattice	5.83098	5.91988	4.49072
56	qpdLattice	6.36946	6.45836	2.25575
58	qpfLattice	6.55775	6.64665	1.67186
61	qpfLattice	7.32562	7.41452	0.02487
63	qpdLattice	7.46992	7.55882	0.00100
70	qpfLattice	8.10749	8.19639	4.49146
76	qpdLattice	8.60769	8.69659	1.59614

```

78 qpfLattice  8.73278  8.82168    3.50000
80 qpdLattice  8.85788  8.94678    1.59614
87 qpfLattice  9.62889  9.71779    3.81660
93 qpdLattice 10.21208 10.30098    1.59614
95 qpfLattice 10.60921 10.69811    3.08092
97 qpdLattice 10.73621 10.82511    1.59614

```

FELSIM_S1_CURRENTS: 26 quadrupoles loaded
time: 672 ms (started: 2026-06-05 15:05:19 -10:00)

```

[6]: %load_ext autotime

#   Confirm key reference positions
s = 0.0
for i, e in enumerate(beamline_full):
    s += float(getattr(e, 'length', 0.0) or 0.0)
    if i in (8, 9, 15, 25, 32, 37, 55, 59, 92, 117):
        print(f"  el {i:3d}  {e.__class__.__name__:20s}  s_end={s:.5f} m")

```

The autotime extension is already loaded. To reload it, use:

```

%reload_ext autotime
el   8  dipole_wedge           s_end=1.80682 m
el   9  driftLattice           s_end=2.03304 m
el  15  driftLattice           s_end=2.73535 m
el  25  dipole_wedge           s_end=3.34855 m
el  32  driftLattice           s_end=4.15187 m
el  37  qpdLattice             s_end=4.52017 m
el  55  driftLattice           s_end=6.36946 m
el  59  driftLattice           s_end=7.11013 m
el  92  driftLattice           s_end=10.21208 m
el 117  driftLattice           s_end=12.39158 m
time: 0 ns (started: 2026-06-05 15:05:19 -10:00)

```

0.3 3 — Statistical Benchmark Infrastructure

`run_benchmark()` runs the same scenario `N_RUNS` times from different random starting currents. `plot_stat_convergence()` plots **mean $\pm$ 1 RMS bands** — the same visual style as Eremey's reference convergence figures.

```

[7]: %load_ext autotime

import time
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

```

```

def method_label(method, label=None):
    if label:
        return label
    if isinstance(method, str):
        return method
    return getattr(method, "__name__", str(method))

# Benchmark runner
def run_benchmark(scenario_name, beamline_slice_len, particles, seg_var, obj,
    ↪ bounds,
                    method, n_runs=5, SEED=42, seed_offset=0, jac=None,
    ↪ options=None, verbose=True, method_tag=None):
    """Run beamOptimizer.calc() n_runs times from uniform-random starting
    ↪ currents."""
    var_names = list({seg_var[i][0] for i in seg_var})
    results = []
    method_name = method_label(method, method_tag)

    for run_i in range(n_runs):
        bl = ExcelElements(EXCEL_PATH).create_beamline()[0:beamline_slice_len]
        p = particles.clone()
        rng = np.random.default_rng(SEED + seed_offset + run_i * 997)
        sp = {v: {"bounds": bounds[v],
                  "start": float(rng.uniform(bounds[v][0], bounds[v][1]))}
              for v in var_names}
        start_x = {v: sp[v]["start"] for v in var_names}
        obj_copy = copy.deepcopy(obj)
        opti = beamOptimizer(bl, p)
        t0 = time.perf_counter()

        try:
            res = opti.calc(method, seg_var, sp, obj_copy, jac=jac,
    ↪ options=options,
                                printResults=False, plotProgress=False)
            wall = time.perf_counter() - t0

            result_x_dict = dict(zip(var_names, res.x))
            i_1 = result_x_dict.get("I", np.nan)
            i_3 = result_x_dict.get("I2", np.nan)

            def safe_get_measured(idx):
                if idx in opti.objectives and len(opti.objectives[idx]) > 0:
                    val = opti.objectives[idx][0].get("measured", np.nan)
                    return float(val.item() if hasattr(val, 'item') else val)
                return np.nan

            alpha_x = safe_get_measured(8)

```

```

        alpha_y = safe_get_measured(9)

        results.append({
            "run": run_i+1, "method": method_name, "scenario":_
↪scenario_name,
            "final_mse": float(res.fun),
            "nfev": int(getattr(res, 'nfev', len(opti.plotMSE))),
            "nit": int(getattr(res, 'nit', -1)),
            "wall_time": round(wall,3),
            "mse_curve": list(opti.plotMSE), "success": bool(res.success),
            "start_x": start_x, "result_x": result_x_dict,
            "I_1": i_1, "I_3": i_3, "alpha_x": alpha_x, "alpha_y": alpha_y
        })
    except Exception as exc:
        wall = time.perf_counter() - t0
        results.append({
            "run": run_i+1, "method": method_name, "scenario":_
↪scenario_name,
            "final_mse": float("nan"), "nfev": -1, "nit": -1,
            "wall_time": round(wall,3), "mse_curve": [], "success": False,
            "start_x": start_x, "result_x": {}, "error": str(exc),
            "I_1": np.nan, "I_3": np.nan, "alpha_x": np.nan, "alpha_y": np.
↪nan
        })
        if verbose: print(f" {method_name} run {run_i+1} FAILED: {exc}")
        continue
    if verbose:
        print(f" {method_name:22s} run {run_i+1}/{n_runs} "
              f"MSE={results[-1]['final_mse']:.3e} nit={results[-1]['nit']}:
↪<3} "
              f"nfev={results[-1]['nfev']:<4} t={wall:.2f}s")
    return results

def results_to_df(results_list):
    keep = ('run', 'method', 'scenario', 'method_tag', 'final_mse', 'nfev', 'nit',
            'wall_time', 'success', 'I_1', 'I_3', 'alpha_x', 'alpha_y')
    return pd.DataFrame([k: r[k] for k in keep if k in r for r in_
↪results_list])

def plot_stat_convergence(results_by_tag, title="Convergence", figsize=None,
                           colors=None, ax=None, save_path=None, scale="log",
                           convergence_epsilon=None):
    standalone = ax is None
    if standalone: fig, ax = plt.subplots(figsize=figsize or (7, 4.5))
    else: fig = ax.get_figure()

    # FIX 1: Matplotlib API Update

```



```

try:
    cmap = plt.colormaps.get_cmap("tab10")
except AttributeError:
    cmap = plt.get_cmap("tab10")

if colors is None: colors = [cmap(i) for i in range(len(results_by_tag))]

legend_handles = []
for (tag, res_list), col in zip(results_by_tag.items(), colors):
    n_t = len(res_list)

    if convergence_epsilon is None:
        converged_iter = [r for r in res_list if r.get('mse_curve')]
    else:
        converged_iter = [r for r in res_list
                          if r.get('mse_curve')
                          and np.isfinite(r['mse_curve'][-1])
                          and r['mse_curve'][-1] < convergence_epsilon]

    n_s = len(converged_iter)
    curves = [r['mse_curve'] for r in converged_iter]

    if not curves:
        legend_handles.append(Line2D([0], [0], color=col, linewidth=2,
                                     label=f"{tag} (0/{n_t} converged)"))
        continue

    max_len = max(len(c) for c in curves)
    padded = np.array([c + [c[-1]] * (max_len - len(c)) for c in curves],
dtype=np.float64)

    # FIX 2: clip to a positive floor so log10 is well-defined
    padded = np.clip(padded, a_min=1e-20, a_max=1e100)

    evals = np.arange(1, max_len + 1)

    log_padded = np.log10(padded)
    log_mean = np.mean(log_padded, axis=0)
    log_std = np.std(log_padded, axis=0)
    mean = 10 ** log_mean
    band_lower = 10 ** (log_mean - log_std)
    band_upper = 10 ** (log_mean + log_std)

    ax.plot(evals, mean, color=col, linewidth=2.0, zorder=3)
    ax.fill_between(evals, np.maximum(band_lower, 1e-20), band_upper,
                    color=col, alpha=0.20, zorder=2)

```

```

        legend_handles.append(Line2D([0], [0], color=col, linewidth=2,
                                     label=f"{tag}  ({n_s}/{n_t} converged)"))

    ax.set_yscale(scale)
    ax.set_xlabel('Function evaluations (Forward Passes)', fontsize=11)
    ax.set_ylabel('Normalised MSE', fontsize=11)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.legend(handles=legend_handles, fontsize=8, framealpha=0.9, loc='upper_
↪right')
    ax.grid(True, which='both', alpha=0.25)
    ax.tick_params(labelsize=9)
    if standalone:
        fig.tight_layout()
        if save_path: fig.savefig(save_path, dpi=200, bbox_inches='tight')
        plt.show()
    return ax
print("Benchmark helpers loaded (Warnings Suppressed).")

```

The autotime extension is already loaded. To reload it, use:

```
%reload_ext autotime
```

Benchmark helpers loaded (Warnings Suppressed).

time: 16 ms (started: 2026-06-05 15:05:19 -10:00)

0.4 4 — Scenario A: Doublet (2 quadrupoles)

Exact replication of Stage 1 from `beamline_optimization.ipynb`: $_x = _y = 0$ at elements 8–9 (s 0.73–1.0 m).

Simple 2D problem — validates all methods and compares FELsim vs COSY timing.

```

[8]: def run_scenario_A(CURRENT_BOUNDS, EPSILON, N_RUNS_A, REF_I, A_OBJ,
↪A_BEAMLINE_LEN, METHODS_A, SEED=42, scale="log", options=None,
↪METHOD_OPTIONS={}):
    A_BOUNDS = {"I1": CURRENT_BOUNDS, "I2": CURRENT_BOUNDS}
    REF_I_1 = REF_I[0]
    REF_I_3 = REF_I[1]
    results_A = {}
    for method_spec in METHODS_A:
        if len(method_spec) == 3:
            method, jac, label = method_spec
        else:
            method, jac = method_spec
            label = None
        options = METHOD_OPTIONS.get(method_label(method, label), None)
        method_name = method_label(method, label)
        tag = method_name + (" +jac" if jac else "")
        print(f"\n Scenario A - {tag}  ({N_RUNS_A} runs)")

```

```

    # Execute benchmark (Note: passing 'jac' here prepares for PyTorch
    ↪ analytical gradients later)
    res = run_benchmark(
        "A",
        A_BEAMLINE_LEN,
        PARTICLES,
        A_VARS,
        A_OBJ,
        A_BOUNDS,
        method=method,
        n_runs=N_RUNS_A,
        SEED=SEED,
        seed_offset=0,
        jac=jac,
        options=options,
        method_tag=method_name,
    )
    for r in res:
        r["method_tag"] = tag
    results_A[tag] = res

# Flatten and merge nested results into a single DataFrame
df_A = results_to_df([r for v in results_A.values() for r in v])

#
    ↪ =====
    # CORE FIX: Patch the bug where derivative-free algorithms return
    ↪ negative iterations
    #
    ↪ =====
    # Since derivative-free algorithms lack internal line searches, their number
    # of function evaluations (nfev) is mathematically equivalent to their true
    ↪ exploration steps (nit).
    # If iteration count < 0 is detected, directly extract 'nfev' as the true
    ↪ iteration count!
    if 'nit' in df_A.columns and 'nfev' in df_A.columns:
        df_A.loc[df_A['nit'] < 0, 'nit'] = df_A.loc[df_A['nit'] < 0, 'nfev']

    print("\n" + "="*115)
    print(f"    SCENARIO A: Robustness (Tol < {EPSILON}) & Discovered Physics
    ↪ State vs Reference")
    print("="*115)

    # 1. Core Evaluation: Check if each run successfully converged below the
    ↪ target MSE threshold

```

```

df_A['is_converged'] = df_A['final_mse'] < EPSILON

# 2. Calculate average performance metrics
def _geom_mean_converged(s):
    conv = s[s < EPSILON]
    return 10 ** np.log10(conv).mean() if len(conv) else np.nan
# ( Note: The redundant 'nfev_mean' has been completely removed as
↳ requested)
summary_stats = df_A.groupby('method_tag').agg(
    conv_rate=('is_converged', 'mean'),
    nit_mean=('nit', 'mean'),
    nfev_mean=('nfev', 'mean'),
    wall_time_mean=('wall_time', 'mean'),
).join(
    df_A.groupby('method_tag')['final_mse']
    .apply(_geom_mean_converged)
    .rename('final_mse_mean')
)

# 3. Extract the Champion Solution: Find the run with the lowest error for
↳ each method,
# extracting its exact hardware currents and resulting alpha observables.
idx_best = df_A.groupby('method_tag')['final_mse'].idxmin()
best_runs = df_A.loc[idx_best, ['method_tag', 'I_1', 'I_3', 'alpha_x',
↳ 'alpha_y']].set_index('method_tag')

# 4. Merge all statistical data
final_summary = summary_stats.join(best_runs).reset_index()

# Inject the expert's gold-standard values to establish the Ground Truth
final_summary['I_1_ref'] = REF_I_1
final_summary['I_3_ref'] = REF_I_3

# Reorder columns: Place the AI-predicted currents right next to the
↳ expert's Reference values
final_summary = final_summary[[
    'method_tag', 'conv_rate', 'nit_mean', 'nfev_mean', 'wall_time_mean',
    'final_mse_mean', 'I_1_ref', 'I_1', 'I_3_ref', 'I_3', 'alpha_x',
↳ 'alpha_y'
]]

# 5. Plain text formatted output to bypass Jupyter HTML rendering bugs
↳ (missing tables)
format_dict = {
    'conv_rate': '{:.0%}',
    'nit_mean': '{:.1f}',

```

```

        'nfev_mean': '{:.1f}',
        'wall_time_mean': '{:.3f} s',
        'final_mse_mean': '{:.2e}',
        'I_1_ref': '{:.4f} A',
        'I_1': '{:.4f} A',
        'I_3_ref': '{:.4f} A',
        'I_3': '{:.4f} A',
        'alpha_x': '{:.2e}',
        'alpha_y': '{:.2e}'
    }

    formatted_df = final_summary.copy()

    # Apply formatting conversions in bulk
    for col, fmt in format_dict.items():
        if col in formatted_df.columns:
            formatted_df[col] = formatted_df[col].apply(lambda x: fmt.format(x)
↪if pd.notna(x) else "NaN")

    # Print the final beautifully aligned data table
    print(formatted_df.to_string(index=False, justify='center'))
    print("="*115)

    # Plot the final algorithm stability curve
    plot_stat_convergence(
        results_A,
        title=f"Scenario A: Convergence Stability (Target MSE < {EPSILON})",
        convergence_epsilon=EPSILON,
        scale=scale
    )
    return results_A, df_A, final_summary

```

time: 16 ms (started: 2026-06-05 15:05:19 -10:00)

```

[9]: %load_ext autotime
def display_results(results_A):

    # Scenario A - Best optimised quadrupole currents per method
    print(" Scenario A: Best Optimised Quadrupole Currents per Method ")
    print(f"{'Method':<22} {'Best MSE':>12} {'I (A)':>10} {'I2 (A)':>10}")
    print(" " * 58)
    for tag, res_list in results_A.items():
        valid = [r for r in res_list
                  if r.get('result_x') and not np.isnan(r['final_mse'])]
        if not valid:
            print(f" {tag:<20} no valid run")
            continue

```

```

        best = min(valid, key=lambda r: r['final_mse'])
        I = best['result_x'].get('I', float('nan'))
        I2 = best['result_x'].get('I2', float('nan'))
        print(f"    {tag:<20} {best['final_mse']:>12.4e} {I:>10.4f} {I2:>10.4f}")
    print()
    print(f" Reference FELsim S1 currents: I={REF_I[0]:.4f} A, I2={REF_I[1]:.4f} A")
    # Mean best across all runs
    all_valid = [r for v in results_A.values() for r in v
                  if r.get('result_x') and not np.isnan(r['final_mse'])]
    if all_valid:
        best_overall = min(all_valid, key=lambda r: r['final_mse'])
        print(f"\n Overall best across all methods/runs:")
        print(f"    Method: {best_overall['method_tag']}, MSE={best_overall['final_mse']:.4e}")
        print(f"    I={best_overall['result_x']['I']:.5f} A, I2={best_overall['result_x']['I2']:.5f} A")

```

The autotime extension is already loaded. To reload it, use:

```
%reload_ext autotime
```

time: 0 ns (started: 2026-06-05 15:05:19 -10:00)

```

[19]: # check default setting for each method
import inspect

import scipy.optimize._lbfgsb_py as lbfgsb
import scipy.optimize._optimize as optimize_core
import scipy.optimize._slsqp_py as slsqp
import scipy.optimize._cobyla_py as cobyla
from scipy.optimize._trustregion_constr import minimize_trustregion_constr as trust_constr

MINIMIZER_FUNCS = {
    "L-BFGS-B": lbfgsb._minimize_lbfgsb,
    "Nelder-Mead": optimize_core._minimize_neldermead,
    "SLSQP": slsqp._minimize_slsqp,
    "trust-constr": trust_constr._minimize_trustregion_constr,
    "COBYLA": cobyla._minimize_cobyla,
}

for method_name, minimize_func in MINIMIZER_FUNCS.items():
    print(f"\n## {method_name}")
    print(minimize_func.__doc__)
    print(inspect.signature(minimize_func))

```

```
## L-BFGS-B
```

Minimize a scalar function of one or more variables using the L-BFGS-B algorithm.

Options

`disp` : None or int

Deprecated option that previously controlled the text printed on the screen during the problem solution. Now the code does not emit any output and this keyword has no function.

.. deprecated:: 1.15.0

This keyword is deprecated and will be removed from SciPy 1.18.0.

`maxcor` : int

The maximum number of variable metric corrections used to define the limited memory matrix. (The limited memory BFGS method does not store the full hessian but uses this many terms in an approximation to it.)

`ftol` : float

The iteration stops when $\|(f^k - f^{k+1}) / \max\{|f^k|, |f^{k+1}|, 1\}\| \leq \text{ftol}$.

`gtol` : float

The iteration will stop when $\max\{|\text{proj } g_i| \mid i = 1, \dots, n\} \leq \text{gtol}$ where $\text{proj } g_i$ is the i -th component of the projected gradient.

`eps` : float or ndarray

If `jac` is None the absolute step size used for numerical approximation of the jacobian via forward differences.

`maxfun` : int

Maximum number of function evaluations before minimization terminates. Note that this function may violate the limit if the gradients are evaluated by numerical differentiation.

`maxiter` : int

Maximum number of algorithm iterations.

`iprint` : int, optional

Deprecated option that previously controlled the text printed on the screen during the problem solution. Now the code does not emit any output and this keyword has no function.

.. deprecated:: 1.15.0

This keyword is deprecated and will be removed from SciPy 1.18.0.

`maxls` : int, optional

Maximum number of line search steps (per iteration). Default is 20.

`finite_diff_rel_step` : None or array_like, optional

If `jac` in ['2-point', '3-point', 'cs'] the relative step size to use for numerical approximation of the jacobian. The absolute step size is computed as $h = \text{rel_step} * \text{sign}(x) * \max(1, \text{abs}(x))$,

possibly adjusted to fit into the bounds. For ``method='3-point'`` the sign of `h` is ignored. If None (default) then step is selected automatically.

workers : int, map-like callable, optional

A map-like callable, such as `multiprocessing.Pool.map` for evaluating any numerical differentiation in parallel.

This evaluation is carried out as ``workers(fun, iterable)``.

.. versionadded:: 1.16.0

Notes

The option `ftol` is exposed via the `scipy.optimize.minimize` interface, but calling `scipy.optimize.fmin\_l\_bfgs\_b` directly exposes `factr`. The relationship between the two is ``ftol = factr \* numpy.finfo(float).eps``. I.e., `factr` multiplies the default machine floating-point precision to arrive at `ftol`.

If the minimization is slow to converge the optimizer may halt if the total number of function evaluations exceeds `maxfun`, or the number of algorithm iterations has reached `maxiter` (whichever comes first). If this is the case then ``result.success=False``, and an appropriate error message is contained in ``result.message``.

```
(fun, x0, args=(), jac=None, bounds=None, disp=<object object at
0x000002AC4225E970>, maxcor=10, ftol=2.220446049250313e-09, gtol=1e-05,
eps=1e-08, maxfun=15000, maxiter=15000, iprint=<object object at
0x000002AC4225E970>, callback=None, maxls=20, finite_diff_rel_step=None,
workers=None, **unknown_options)
```

Nelder-Mead

Minimization of scalar function of one or more variables using the Nelder-Mead algorithm.

Options

disp : bool

Set to True to print convergence messages.

maxiter, maxfev : int

Maximum allowed number of iterations and function evaluations.

Will default to ``N\*200``, where ``N`` is the number of variables, if neither `maxiter` or `maxfev` is set. If both `maxiter` and `maxfev` are set, minimization will stop at the first reached.

return_all : bool, optional

Set to True to return a list of the best solution at each of the iterations.

`initial_simplex` : array_like of shape (N + 1, N)
 Initial simplex. If given, overrides `x0`.
`initial_simplex[j,:]` should contain the coordinates of
 the jth vertex of the `N+1` vertices in the simplex, where
`N` is the dimension.

`xatol` : float, optional
 Absolute error in `xopt` between iterations that is acceptable for
 convergence.

`fatol` : number, optional
 Absolute error in `func(xopt)` between iterations that is acceptable for
 convergence.

`adaptive` : bool, optional
 Adapt algorithm parameters to dimensionality of problem. Useful for
 high-dimensional minimization [1]_.

`bounds` : sequence or `Bounds`, optional
 Bounds on variables. There are two ways to specify the bounds:

1. Instance of `Bounds` class.
2. Sequence of `(min, max)` pairs for each element in `x`. None
 is used to specify no bound.

Note that this just clips all vertices in simplex based on
 the bounds.

References

.. [1] Gao, F. and Han, L.
 Implementing the Nelder-Mead simplex algorithm with adaptive
 parameters. 2012. Computational Optimization and Applications.
 51:1, pp. 259-277

```
(func, x0, args=(), callback=None, maxiter=None, maxfev=None, disp=False,
return_all=False, initial_simplex=None, xatol=0.0001, fatol=0.0001,
adaptive=False, bounds=None, **unknown_options)
```

SLSQP

Minimize a scalar function of one or more variables using Sequential
 Least Squares Programming (SLSQP).

Parameters

`ftol` : float
 Precision target for the value of `f` in the stopping criterion. This
 value controls the final accuracy for checking various optimality conditions;
 gradient of the lagrangian and absolute sum of the constraint violations

should be lower than ``ftol``. Similarly, computed step size and the objective function changes are checked against this value. Default is 1e-6.

`eps` : float
Step size used for numerical approximation of the Jacobian.

`disp` : bool
Set to True to print convergence messages. If False, ``verbosity`` is ignored and set to 0.

`maxiter` : int
Maximum number of iterations.

`finite_diff_rel_step` : None or array_like, optional
If ``jac in ['2-point', '3-point', 'cs']`` the relative step size to use for numerical approximation of ``jac``. The absolute step size is computed as ``h = rel\_step \* sign(x) \* max(1, abs(x))``, possibly adjusted to fit into the bounds. For ``method='3-point'`` the sign of ``h`` is ignored. If None (default) then step is selected automatically.

`workers` : int, map-like callable, optional
A map-like callable, such as ``multiprocessing.Pool.map`` for evaluating any numerical differentiation in parallel.
This evaluation is carried out as ``workers(fun, iterable)``.

.. versionadded:: 1.16.0

Returns

`res` : `OptimizeResult`
The optimization result represented as an ``OptimizeResult`` object.
In this dict-like object the following fields are of particular importance:

for ``x`` the solution array, ``success`` a Boolean flag indicating if the optimizer exited successfully, ``message`` which describes the reason for termination, and ``multipliers`` which contains the Karush-Kuhn-Tucker (KKT) multipliers for the QP approximation used in solving the original nonlinear problem. See ``Notes`` below. See also ``OptimizeResult`` for a description of other attributes.

Notes

The KKT multipliers are returned in the ``OptimizeResult.multipliers`` attribute as a NumPy array. Denoting the dimension of the equality constraints with ``meq``, and of inequality constraints with ``mineq``, then the returned array slice ``m[:meq]`` contains the multipliers for the equality constraints, and the remaining ``m[meq:meq + mineq]`` contains the multipliers for the

inequality constraints. The multipliers corresponding to bound inequalities are not returned. See [1]_ pp. 321 or [2]_ for an explanation of how to interpret

these multipliers. The internal QP problem is solved using the methods given in [3]_ Chapter 25.

Note that if new-style ``NonlinearConstraint`` or ``LinearConstraint`` were used, then ``minimize`` converts them first to old-style constraint dicts. It is possible for a single new-style constraint to simultaneously contain both inequality and equality constraints. This means that if there is mixing within a single constraint, then the returned list of multipliers will have a different length than the original new-style constraints.

References

- .. [1] Nocedal, J., and S J Wright, 2006, "Numerical Optimization", Springer, New York.
- .. [2] Kraft, D., "A software package for sequential quadratic programming", 1988, Tech. Rep. DFVLR-FB 88-28, DLR German Aerospace Center, Germany.
- .. [3] Lawson, C. L., and R. J. Hanson, 1995, "Solving Least Squares Problems", SIAM, Philadelphia, PA.

```
(func, x0, args=(), jac=None, bounds=None, constraints=(), maxiter=100,
ftol=1e-06, iprint=1, disp=False, eps=np.float64(1.4901161193847656e-08),
callback=None, finite_diff_rel_step=None, workers=None, **unknown_options)
```

```
## trust-constr
```

Minimize a scalar function subject to constraints.

Parameters

`gtol` : float, optional

Tolerance for termination by the norm of the Lagrangian gradient. The algorithm will terminate when both the infinity norm (i.e., max abs value) of the Lagrangian gradient and the constraint violation are smaller than ``gtol``. Default is 1e-8.

`xtol` : float, optional

Tolerance for termination by the change of the independent variable. The algorithm will terminate when ``tr_radius < xtol``, where ``tr_radius`` is the radius of the trust region used in the algorithm. Default is 1e-8.

`barrier_tol` : float, optional

Threshold on the barrier parameter for the algorithm termination. When inequality constraints are present, the algorithm will terminate only when the barrier parameter is less than ``barrier_tol``.

Default is 1e-8.

`sparse_jacobian` : {bool, None}, optional
 Determines how to represent Jacobians of the constraints. If bool, then Jacobians of all the constraints will be converted to the corresponding format. If None (default), then Jacobians won't be converted, but the algorithm can proceed only if they all have the same format.

`initial_tr_radius`: float, optional
 Initial trust radius. The trust radius gives the maximum distance between solution points in consecutive iterations. It reflects the trust the algorithm puts in the local approximation of the optimization problem. For an accurate local approximation the trust-region should be large and for an approximation valid only close to the current point it should be a small one. The trust radius is automatically updated throughout the optimization process, with ``initial\_tr\_radius`` being its initial value.

Default is 1 (recommended in [1]_, p. 19).

`initial_constr_penalty` : float, optional
 Initial constraints penalty parameter. The penalty parameter is used for balancing the requirements of decreasing the objective function and satisfying the constraints. It is used for defining the merit function:

$$\text{merit_function}(x) = \text{fun}(x) + \text{constr_penalty} * \text{constr_norm_l2}(x)$$

where ``constr\_norm\_l2(x)`` is the l2 norm of a vector containing all the constraints. The merit function is used for accepting or rejecting trial points and ``constr\_penalty`` weights the two conflicting goals of reducing objective function and constraints. The penalty is automatically updated throughout the optimization process, with ``initial\_constr\_penalty`` being its initial value. Default is 1 (recommended in [1]_, p 19).

`initial_barrier_parameter, initial_barrier_tolerance`: float, optional
 Initial barrier parameter and initial tolerance for the barrier subproblem.

Both are used only when inequality constraints are present. For dealing with optimization problems $\min_x f(x)$ subject to inequality constraints $c(x) \leq 0$ the algorithm introduces slack variables, solving the problem $\min_{(x,s)} f(x) + \text{barrier_parameter} * \sum(\ln(s))$ subject to the equality constraints $c(x) + s = 0$ instead of the original problem. This subproblem is solved for decreasing values of ``barrier\_parameter`` and with decreasing tolerances for the termination, starting with ``initial\_barrier\_parameter``

for the barrier parameter and ``initial\_barrier\_tolerance`` for the barrier tolerance. Default is 0.1 for both values (recommended in [1]_ p. 19).

Also note that ``barrier\_parameter`` and ``barrier\_tolerance`` are updated

with the same prefactor.

`factorization_method` : string or None, optional

Method to factorize the Jacobian of the constraints. Use None (default) for the auto selection or one of:

- 'NormalEquation' (requires scikit-sparse)
- 'AugmentedSystem'
- 'QRFactorization'
- 'SVDFactorization'

The methods 'NormalEquation' and 'AugmentedSystem' can be used only with sparse constraints. The projections required by the algorithm will be computed using, respectively, the normal equation and the augmented system approaches explained in [1]_. 'NormalEquation' computes the Cholesky factorization of $A^T A$ and 'AugmentedSystem' performs the LU factorization of an augmented system. They usually provide similar results. 'AugmentedSystem' is used by default for sparse matrices.

The methods 'QRFactorization' and 'SVDFactorization' can be used only with dense constraints. They compute the required projections using, respectively, QR and SVD factorizations. The 'SVDFactorization' method can cope with Jacobian matrices with deficient row rank and will be used whenever other factorization methods fail (which may imply the conversion of sparse matrices to a dense format when required).

By default, 'QRFactorization' is used for dense matrices.

`finite_diff_rel_step` : None or array_like, optional

Relative step size for the finite difference approximation.

`maxiter` : int, optional

Maximum number of algorithm iterations. Default is 1000.

`verbose` : {0, 1, 2, 3}, optional

Level of algorithm's verbosity:

- * 0 (default) : work silently.
- * 1 : display a termination report.
- * 2 : display progress during iterations.
- * 3 : display progress during iterations (more complete report).

`disp` : bool, optional

If True (default), then `verbose` will be set to 1 if it was 0.

`workers` : int, map-like callable, optional

A map-like callable, such as ``multiprocessing.Pool.map`` for evaluating any numerical differentiation in parallel.

This evaluation is carried out as ``workers(fun, iterable)``.

.. versionadded:: 1.16.0

Returns

`OptimizeResult` with the fields documented below. Note the following:

1. All values corresponding to the constraints are ordered as they were passed to the solver. And values corresponding to `bounds` constraints are put *after* other constraints.
2. All numbers of function, Jacobian or Hessian evaluations correspond to numbers of actual Python function calls. It means, for example, that if a Jacobian is estimated by finite differences, then the number of Jacobian evaluations will be zero and the number of function evaluations will be incremented by all calls during the finite difference estimation.

`x` : ndarray, shape (n,)
Solution found.

`optimality` : float
Infinity norm of the Lagrangian gradient at the solution.

`constr_violation` : float
Maximum constraint violation at the solution.

`fun` : float
Objective function at the solution.

`grad` : ndarray, shape (n,)
Gradient of the objective function at the solution.

`lagrangian_grad` : ndarray, shape (n,)
Gradient of the Lagrangian function at the solution.

`nit` : int
Total number of iterations.

`nfev` : integer
Number of the objective function evaluations.

`njev` : integer
Number of the objective function gradient evaluations.

`nhev` : integer
Number of the objective function Hessian evaluations.

`cg_niter` : int
Total number of the conjugate gradient method iterations.

`method` : {'equality_constrained_sqp', 'tr_interior_point'}
Optimization method used.

`constr` : list of ndarray
List of constraint values at the solution.

`jac` : list of {ndarray, sparse array}
List of the Jacobian matrices of the constraints at the solution.

`v` : list of ndarray
List of the Lagrange multipliers for the constraints at the solution.

For an inequality constraint a positive multiplier means that the upper bound is active, a negative multiplier means that the lower bound is active and if a multiplier is zero it means the constraint is not active.

constr_nfev : list of int

Number of constraint evaluations for each of the constraints.

constr_njev : list of int

Number of Jacobian matrix evaluations for each of the constraints.

constr_nhev : list of int

Number of Hessian evaluations for each of the constraints.

tr_radius : float

Radius of the trust region at the last iteration.

constr_penalty : float

Penalty parameter at the last iteration, see `initial\_constr\_penalty`.

barrier_tolerance : float

Tolerance for the barrier subproblem at the last iteration.

Only for problems with inequality constraints.

barrier_parameter : float

Barrier parameter at the last iteration. Only for problems with inequality constraints.

execution_time : float

Total execution time.

message : str

Termination message.

status : {0, 1, 2, 3, 4}

Termination status:

* 0 : The maximum number of function evaluations is exceeded.

* 1 : `gtol` termination condition is satisfied.

* 2 : `xtol` termination condition is satisfied.

* 3 : `callback` function requested termination.

* 4 : Constraint violation exceeds 'gtol'.

.. versionchanged:: 1.15.0

If the constraint violation exceeds `gtol`, then ``result.success`` will now be False.

cg_stop_cond : int

Reason for CG subproblem termination at the last iteration:

* 0 : CG subproblem not evaluated.

* 1 : Iteration limit was reached.

* 2 : Reached the trust-region boundary.

* 3 : Negative curvature detected.

* 4 : Tolerance was satisfied.

References

.. [1] Conn, A. R., Gould, N. I., & Toint, P. L.
Trust region methods. 2000. Siam. pp. 19.

```
(fun, x0, args, grad, hess, hessp, bounds, constraints, xtol=1e-08, gtol=1e-08,
barrier_tol=1e-08, sparse_jacobian=None, callback=None, maxiter=1000, verbose=0,
finite_diff_rel_step=None, initial_constr_penalty=1.0, initial_tr_radius=1.0,
initial_barrier_parameter=0.1, initial_barrier_tolerance=0.1,
factorization_method=None, disp=False, workers=None)
```

COBYLA

Minimize a scalar function of one or more variables using the
Constrained Optimization BY Linear Approximation (COBYLA) algorithm.
This method uses the pure-python implementation of the algorithm from PRIMA.

Options

rhobeg : float

Reasonable initial changes to the variables.

tol : float

Final accuracy in the optimization (not precisely guaranteed).

This is a lower bound on the size of the trust region.

disp : int

Controls the frequency of output:

0. (default) There will be no printing

1. A message will be printed to the screen at the end of iteration,

showing

the best vector of variables found and its objective function

value

2. in addition to 1, each new value of RHO is printed to the screen,

with the best vector of variables so far and its objective

function

value.

3. in addition to 2, each function evaluation with its variables

will

be printed to the screen.

maxiter : int

Maximum number of function evaluations.

catol : float

Tolerance (absolute) for constraint violations

f_target : float

Stop if the objective function is less than `f\_target`.

.. versionchanged:: 1.16.0

The original Powell implementation was replaced by a pure
Python version from the PRIMA package, with bug fixes and
improvements being made.

References

Zhang Z. (2023), "PRIMA: Reference Implementation for Powell's Methods with Modernization and Amelioration", <https://www.libprima.net>,
:doi:`10.5281/zenodo.8052654`

```
(fun, x0, args=(), constraints=(), rhobeg=1.0, tol=0.0001, maxiter=1000, disp=0,  
catol=None, f_target=-inf, callback=None, bounds=None, **unknown_options)  
time: 0 ns (started: 2026-06-05 15:18:46 -10:00)
```

```
[22]: CURRENT_BOUNDS = (0.01, 1.5)  
EPSILON = 1e-3  
  
REF_I_1 = 0.8218  
REF_I_3 = 1.0430  
REF_I = [REF_I_1, REF_I_3]  
  
# parameter setting  
N_RUNS_A = 100  
A_BEAMLINE_LEN = 10  
  
# this is default version  
METHOD_OPTIONS = {  
    "L-BFGS-B": {  
        "maxcor": 10,  
        "ftol": 2.220446049250313e-09,  
        "gtol": 1e-5,  
        "eps": 1e-8,  
        "maxfun": 15000,  
        "maxiter": 15000,  
        "maxls": 20,  
    },  
    "SLSQP": {  
        "maxiter": 100,  
        "ftol": 1e-6,  
        "iprint": 1,  
        "disp": False,  
        "eps": 1.4901161193847656e-08,  
        "finite_diff_rel_step": None,  
        "workers": None,  
    },  
    "trust-constr": {  
        "xtol": 1e-8,  
        "gtol": 1e-8,  
        "barrier_tol": 1e-8,  
        "sparse_jacobian": None,
```

```

        "maxiter": 1000,
        "verbose": 0,
        "finite_diff_rel_step": None,
        "initial_constr_penalty": 1.0,
        "initial_tr_radius": 1.0,
        "initial_barrier_parameter": 0.1,
        "initial_barrier_tolerance": 0.1,
        "factorization_method": None,
        "disp": False,
        "workers": None,
    },
    "Nelder-Mead": {
        "maxiter": None,
        "maxfev": None,
        "disp": False,
        "return_all": False,
        "initial_simplex": None,
        "xatol": 1e-4,
        "fatol": 1e-4,
        "adaptive": False,
    },
    "COBYLA": {
        "rhobeg": 1.0,
        "tol": 1e-4,
        "maxiter": 1000,
        "disp": 0,
        "catol": None,
        "f_target": float("-inf"),
    },
},
}

METHODS_A = [
    ("Nelder-Mead", None),
    ("L-BFGS-B", "2-point"),
    ("SLSQP", "2-point"),
    ("COBYLA", None),
    ("trust-constr", None),
]

A_VARS = {
    1: ["I", "current", lambda x: x],
    3: ["I2", "current", lambda x: x],
}

A_OBJ = {
    8: [{"measure": ["x", "alpha"], "goal": 0.0, "weight": 1.0}],
    9: [{"measure": ["y", "alpha"], "goal": 0.0, "weight": 1.0}],
}

```

time: 0 ns (started: 2026-06-05 15:42:07 -10:00)

```
[11]: # test whether float64 works
import numpy as np, copy

print("PARTICLES dtype:", PARTICLES.dtype)

A_BEAMLINE_LEN = 138
bl = ExcelElements(EXCEL_PATH).create_beamline()[:A_BEAMLINE_LEN]
opti = beamOptimizer(bl, PARTICLES.clone())
opti.calc("Nelder-Mead", A_VARS,
          {"I":{"bounds":(0.01,1.5),"start":0.5}, "I2":{"bounds":(0.01,1.
↵5),"start":0.5}},
          copy.deepcopy(A_OBJ), printResults=False)

print("variablesToOptimize:", opti.variablesToOptimize)

x = np.array([0.5, 0.5])
base = opti._optiSpeed(x)
print(f"\nbase MSE = {base:.6e}")
print("test:")
for h in [1e-5, 1e-7, 1e-9, 1e-11]:
    xp = x.copy(); xp[0] += h
    diff = opti._optiSpeed(xp) - base
    print(f"  h={h:.0e}:  ΔMSE={diff:+.3e}    slope {diff/h:+.4e}")
```

PARTICLES dtype: torch.float64
variablesToOptimize: ['I', 'I2']

base MSE = 4.969018e-01

test:

h=1e-05: ΔMSE=-3.885e-05 slope -3.8847e+00

h=1e-07: ΔMSE=-3.885e-07 slope -3.8847e+00

h=1e-09: ΔMSE=-3.885e-09 slope -3.8847e+00

h=1e-11: ΔMSE=-3.885e-11 slope -3.8847e+00

time: 4min 41s (started: 2026-06-05 15:05:20 -10:00)

```
[23]: %load_ext autotime

results_A, df_A, final_summary = run_scenario_A(CURRENT_BOUNDS, EPSILON,↵
↵N_RUNS_A, REF_I, A_OBJ, A_BEAMLINE_LEN,↵
↵METHODS_A,scale="log",METHOD_OPTIONS=METHOD_OPTIONS)
display_results(results_A)
```

The autotime extension is already loaded. To reload it, use:

```
%reload_ext autotime
```

Scenario A - Nelder-Mead (100 runs)

Nelder-Mead	run 1/100	MSE=1.559e-08	nit=44	nfev=84	t=14.00s
Nelder-Mead	run 2/100	MSE=9.229e-09	nit=37	nfev=73	t=13.89s
Nelder-Mead	run 3/100	MSE=9.466e-09	nit=53	nfev=102	t=16.63s
Nelder-Mead	run 4/100	MSE=3.242e-08	nit=27	nfev=53	t=8.71s
Nelder-Mead	run 5/100	MSE=2.836e-09	nit=40	nfev=77	t=12.21s
Nelder-Mead	run 6/100	MSE=2.226e-08	nit=32	nfev=61	t=10.77s
Nelder-Mead	run 7/100	MSE=2.014e-07	nit=44	nfev=85	t=13.08s
Nelder-Mead	run 8/100	MSE=6.463e-10	nit=45	nfev=85	t=12.50s
Nelder-Mead	run 9/100	MSE=3.169e-08	nit=45	nfev=87	t=13.07s
Nelder-Mead	run 10/100	MSE=4.284e-08	nit=43	nfev=80	t=11.82s
Nelder-Mead	run 11/100	MSE=2.780e-08	nit=34	nfev=66	t=9.88s
Nelder-Mead	run 12/100	MSE=5.476e-09	nit=43	nfev=83	t=12.21s
Nelder-Mead	run 13/100	MSE=4.517e-09	nit=46	nfev=87	t=13.19s
Nelder-Mead	run 14/100	MSE=6.055e-08	nit=35	nfev=69	t=10.33s
Nelder-Mead	run 15/100	MSE=4.686e-09	nit=38	nfev=74	t=11.24s
Nelder-Mead	run 16/100	MSE=1.359e-08	nit=26	nfev=52	t=7.77s
Nelder-Mead	run 17/100	MSE=2.295e-09	nit=41	nfev=82	t=12.35s
Nelder-Mead	run 18/100	MSE=1.855e-08	nit=46	nfev=90	t=15.08s
Nelder-Mead	run 19/100	MSE=7.546e-08	nit=33	nfev=64	t=11.37s
Nelder-Mead	run 20/100	MSE=2.903e-09	nit=55	nfev=100	t=15.59s
Nelder-Mead	run 21/100	MSE=1.639e-08	nit=45	nfev=85	t=12.95s
Nelder-Mead	run 22/100	MSE=6.352e-08	nit=30	nfev=58	t=8.79s
Nelder-Mead	run 23/100	MSE=3.752e-09	nit=36	nfev=71	t=10.79s
Nelder-Mead	run 24/100	MSE=6.822e-08	nit=43	nfev=81	t=12.96s
Nelder-Mead	run 25/100	MSE=4.001e-09	nit=47	nfev=89	t=13.11s
Nelder-Mead	run 26/100	MSE=1.771e+01	nit=25	nfev=47	t=7.27s
Nelder-Mead	run 27/100	MSE=3.301e-09	nit=58	nfev=108	t=16.85s
Nelder-Mead	run 28/100	MSE=1.894e-08	nit=36	nfev=67	t=10.65s
Nelder-Mead	run 29/100	MSE=3.147e-09	nit=43	nfev=83	t=12.72s
Nelder-Mead	run 30/100	MSE=5.683e-09	nit=34	nfev=69	t=10.64s
Nelder-Mead	run 31/100	MSE=1.433e-08	nit=38	nfev=68	t=9.87s
Nelder-Mead	run 32/100	MSE=1.714e-08	nit=40	nfev=76	t=11.08s
Nelder-Mead	run 33/100	MSE=1.377e-08	nit=30	nfev=59	t=8.51s
Nelder-Mead	run 34/100	MSE=1.370e-09	nit=45	nfev=85	t=12.21s
Nelder-Mead	run 35/100	MSE=5.410e-09	nit=46	nfev=89	t=12.95s
Nelder-Mead	run 36/100	MSE=2.559e-08	nit=32	nfev=60	t=8.63s
Nelder-Mead	run 37/100	MSE=4.174e-09	nit=43	nfev=81	t=11.80s
Nelder-Mead	run 38/100	MSE=2.799e-09	nit=51	nfev=97	t=14.63s
Nelder-Mead	run 39/100	MSE=5.175e-09	nit=41	nfev=79	t=11.54s
Nelder-Mead	run 40/100	MSE=4.105e-09	nit=43	nfev=79	t=11.31s
Nelder-Mead	run 41/100	MSE=2.244e-08	nit=33	nfev=63	t=9.17s
Nelder-Mead	run 42/100	MSE=1.032e-07	nit=46	nfev=85	t=12.63s
Nelder-Mead	run 43/100	MSE=1.752e-09	nit=37	nfev=70	t=10.43s
Nelder-Mead	run 44/100	MSE=5.904e-02	nit=22	nfev=40	t=5.69s
Nelder-Mead	run 45/100	MSE=1.269e-08	nit=34	nfev=64	t=12.54s
Nelder-Mead	run 46/100	MSE=1.506e-09	nit=55	nfev=105	t=14.74s
Nelder-Mead	run 47/100	MSE=4.378e-09	nit=41	nfev=79	t=11.23s
Nelder-Mead	run 48/100	MSE=8.458e-09	nit=44	nfev=89	t=12.75s

Nelder-Mead	run 49/100	MSE=6.774e-08	nit=47	nfev=89	t=12.69s
Nelder-Mead	run 50/100	MSE=2.398e-08	nit=42	nfev=80	t=11.96s
Nelder-Mead	run 51/100	MSE=2.697e-08	nit=33	nfev=65	t=9.54s
Nelder-Mead	run 52/100	MSE=4.552e-09	nit=43	nfev=79	t=11.44s
Nelder-Mead	run 53/100	MSE=9.923e-09	nit=50	nfev=94	t=13.12s
Nelder-Mead	run 54/100	MSE=1.418e-08	nit=49	nfev=94	t=13.29s
Nelder-Mead	run 55/100	MSE=6.797e-09	nit=53	nfev=99	t=14.06s
Nelder-Mead	run 56/100	MSE=2.403e-08	nit=39	nfev=74	t=10.35s
Nelder-Mead	run 57/100	MSE=5.101e-09	nit=36	nfev=71	t=10.09s
Nelder-Mead	run 58/100	MSE=3.079e-08	nit=37	nfev=71	t=9.95s
Nelder-Mead	run 59/100	MSE=1.152e-09	nit=38	nfev=71	t=9.99s
Nelder-Mead	run 60/100	MSE=1.088e-08	nit=44	nfev=86	t=12.37s
Nelder-Mead	run 61/100	MSE=2.570e-09	nit=54	nfev=103	t=14.57s
Nelder-Mead	run 62/100	MSE=1.936e-08	nit=32	nfev=63	t=8.85s
Nelder-Mead	run 63/100	MSE=6.139e-09	nit=39	nfev=76	t=10.85s
Nelder-Mead	run 64/100	MSE=2.548e-09	nit=46	nfev=90	t=12.80s
Nelder-Mead	run 65/100	MSE=5.569e-09	nit=48	nfev=93	t=13.41s
Nelder-Mead	run 66/100	MSE=5.274e-09	nit=42	nfev=80	t=11.23s
Nelder-Mead	run 67/100	MSE=3.120e-08	nit=46	nfev=87	t=12.13s
Nelder-Mead	run 68/100	MSE=5.108e-08	nit=31	nfev=62	t=8.67s
Nelder-Mead	run 69/100	MSE=4.614e-08	nit=51	nfev=93	t=13.15s
Nelder-Mead	run 70/100	MSE=6.579e-09	nit=28	nfev=55	t=7.81s
Nelder-Mead	run 71/100	MSE=5.904e-02	nit=24	nfev=45	t=6.30s
Nelder-Mead	run 72/100	MSE=8.423e-09	nit=53	nfev=102	t=14.42s
Nelder-Mead	run 73/100	MSE=4.279e-09	nit=34	nfev=67	t=9.42s
Nelder-Mead	run 74/100	MSE=3.162e-08	nit=35	nfev=67	t=9.40s
Nelder-Mead	run 75/100	MSE=8.945e-09	nit=32	nfev=59	t=8.48s
Nelder-Mead	run 76/100	MSE=6.087e-08	nit=41	nfev=78	t=10.89s
Nelder-Mead	run 77/100	MSE=1.062e-08	nit=35	nfev=65	t=9.22s
Nelder-Mead	run 78/100	MSE=3.687e-08	nit=55	nfev=107	t=15.19s
Nelder-Mead	run 79/100	MSE=5.500e-08	nit=32	nfev=62	t=8.90s
Nelder-Mead	run 80/100	MSE=6.047e-02	nit=13	nfev=25	t=3.41s
Nelder-Mead	run 81/100	MSE=1.979e-08	nit=40	nfev=74	t=10.34s
Nelder-Mead	run 82/100	MSE=1.355e-08	nit=34	nfev=64	t=9.09s
Nelder-Mead	run 83/100	MSE=1.419e-08	nit=32	nfev=64	t=8.99s
Nelder-Mead	run 84/100	MSE=7.435e-09	nit=32	nfev=60	t=8.47s
Nelder-Mead	run 85/100	MSE=1.390e-08	nit=53	nfev=101	t=14.20s
Nelder-Mead	run 86/100	MSE=1.982e-09	nit=46	nfev=89	t=12.57s
Nelder-Mead	run 87/100	MSE=8.345e-09	nit=31	nfev=60	t=8.39s
Nelder-Mead	run 88/100	MSE=1.772e-08	nit=33	nfev=64	t=8.96s
Nelder-Mead	run 89/100	MSE=2.778e-09	nit=41	nfev=77	t=10.78s
Nelder-Mead	run 90/100	MSE=2.110e-08	nit=32	nfev=63	t=9.05s
Nelder-Mead	run 91/100	MSE=1.325e-08	nit=31	nfev=59	t=8.26s
Nelder-Mead	run 92/100	MSE=1.594e-08	nit=39	nfev=73	t=10.28s
Nelder-Mead	run 93/100	MSE=1.075e-08	nit=38	nfev=73	t=10.33s
Nelder-Mead	run 94/100	MSE=3.327e-08	nit=37	nfev=72	t=10.29s
Nelder-Mead	run 95/100	MSE=1.400e-08	nit=29	nfev=57	t=8.02s
Nelder-Mead	run 96/100	MSE=4.475e-08	nit=41	nfev=75	t=10.55s

Nelder-Mead	run 97/100	MSE=9.398e-09	nit=36	nfev=70	t=10.01s
Nelder-Mead	run 98/100	MSE=1.918e-08	nit=34	nfev=65	t=9.17s
Nelder-Mead	run 99/100	MSE=6.395e-09	nit=43	nfev=79	t=11.15s
Nelder-Mead	run 100/100	MSE=1.595e-08	nit=31	nfev=58	t=8.14s

Scenario A - L-BFGS-B+jac (100 runs)

L-BFGS-B	run 1/100	MSE=3.398e-13	nit=12	nfev=54	t=7.41s
L-BFGS-B	run 2/100	MSE=1.067e-13	nit=13	nfev=57	t=7.95s
L-BFGS-B	run 3/100	MSE=1.110e-12	nit=10	nfev=45	t=6.34s
L-BFGS-B	run 4/100	MSE=1.492e-13	nit=11	nfev=42	t=5.91s
L-BFGS-B	run 5/100	MSE=2.363e-13	nit=16	nfev=63	t=8.80s
L-BFGS-B	run 6/100	MSE=2.547e-13	nit=7	nfev=39	t=5.53s
L-BFGS-B	run 7/100	MSE=1.194e-13	nit=8	nfev=48	t=6.71s
L-BFGS-B	run 8/100	MSE=1.585e-13	nit=8	nfev=60	t=8.36s
L-BFGS-B	run 9/100	MSE=5.688e-13	nit=11	nfev=42	t=5.92s
L-BFGS-B	run 10/100	MSE=6.121e-13	nit=14	nfev=66	t=9.40s
L-BFGS-B	run 11/100	MSE=1.835e-12	nit=11	nfev=42	t=6.02s
L-BFGS-B	run 12/100	MSE=5.802e-13	nit=13	nfev=45	t=6.34s
L-BFGS-B	run 13/100	MSE=9.578e-13	nit=14	nfev=81	t=11.22s
L-BFGS-B	run 14/100	MSE=8.253e-12	nit=16	nfev=54	t=7.57s
L-BFGS-B	run 15/100	MSE=7.010e-13	nit=7	nfev=51	t=7.15s
L-BFGS-B	run 16/100	MSE=5.810e-13	nit=7	nfev=27	t=3.89s
L-BFGS-B	run 17/100	MSE=9.179e-12	nit=11	nfev=69	t=9.75s
L-BFGS-B	run 18/100	MSE=1.465e-12	nit=10	nfev=51	t=7.22s
L-BFGS-B	run 19/100	MSE=3.933e-13	nit=14	nfev=54	t=7.60s
L-BFGS-B	run 20/100	MSE=1.211e-13	nit=9	nfev=51	t=7.15s
L-BFGS-B	run 21/100	MSE=3.363e-13	nit=11	nfev=57	t=7.94s
L-BFGS-B	run 22/100	MSE=1.958e-13	nit=10	nfev=42	t=5.94s
L-BFGS-B	run 23/100	MSE=1.793e-12	nit=12	nfev=57	t=8.00s
L-BFGS-B	run 24/100	MSE=3.364e-13	nit=15	nfev=75	t=10.44s
L-BFGS-B	run 25/100	MSE=5.523e-13	nit=13	nfev=45	t=6.32s
L-BFGS-B	run 26/100	MSE=8.593e-12	nit=14	nfev=48	t=6.77s
L-BFGS-B	run 27/100	MSE=3.534e-13	nit=12	nfev=72	t=10.07s
L-BFGS-B	run 28/100	MSE=3.426e-13	nit=12	nfev=57	t=7.95s
L-BFGS-B	run 29/100	MSE=1.222e-13	nit=14	nfev=57	t=7.98s
L-BFGS-B	run 30/100	MSE=9.862e-13	nit=11	nfev=45	t=6.33s
L-BFGS-B	run 31/100	MSE=3.849e-12	nit=11	nfev=45	t=6.44s
L-BFGS-B	run 32/100	MSE=3.465e-13	nit=14	nfev=63	t=8.75s
L-BFGS-B	run 33/100	MSE=1.190e-12	nit=12	nfev=45	t=6.39s
L-BFGS-B	run 34/100	MSE=3.006e-13	nit=15	nfev=81	t=11.52s
L-BFGS-B	run 35/100	MSE=6.070e-13	nit=16	nfev=72	t=10.02s
L-BFGS-B	run 36/100	MSE=1.483e-12	nit=15	nfev=54	t=7.58s
L-BFGS-B	run 37/100	MSE=1.771e-12	nit=13	nfev=45	t=6.36s
L-BFGS-B	run 38/100	MSE=5.505e-13	nit=15	nfev=54	t=7.58s
L-BFGS-B	run 39/100	MSE=1.720e-13	nit=14	nfev=57	t=8.00s
L-BFGS-B	run 40/100	MSE=3.013e-13	nit=14	nfev=54	t=7.78s
L-BFGS-B	run 41/100	MSE=6.065e-13	nit=8	nfev=51	t=7.20s
L-BFGS-B	run 42/100	MSE=1.656e-13	nit=10	nfev=39	t=5.39s

L-BFGS-B	run 43/100	MSE=1.816e-12	nit=10	nfev=39	t=5.47s
L-BFGS-B	run 44/100	MSE=6.431e-13	nit=13	nfev=45	t=6.46s
L-BFGS-B	run 45/100	MSE=3.250e-13	nit=10	nfev=45	t=6.42s
L-BFGS-B	run 46/100	MSE=4.067e-12	nit=10	nfev=45	t=6.30s
L-BFGS-B	run 47/100	MSE=2.746e-12	nit=15	nfev=93	t=13.04s
L-BFGS-B	run 48/100	MSE=1.395e-13	nit=14	nfev=48	t=6.84s
L-BFGS-B	run 49/100	MSE=1.320e-13	nit=10	nfev=39	t=5.57s
L-BFGS-B	run 50/100	MSE=3.402e-13	nit=13	nfev=57	t=8.32s
L-BFGS-B	run 51/100	MSE=4.635e-13	nit=13	nfev=54	t=7.59s
L-BFGS-B	run 52/100	MSE=6.642e-14	nit=7	nfev=54	t=7.58s
L-BFGS-B	run 53/100	MSE=1.799e-12	nit=14	nfev=66	t=9.20s
L-BFGS-B	run 54/100	MSE=1.136e-13	nit=11	nfev=54	t=7.55s
L-BFGS-B	run 55/100	MSE=1.288e-13	nit=14	nfev=54	t=7.61s
L-BFGS-B	run 56/100	MSE=6.758e-12	nit=7	nfev=36	t=5.09s
L-BFGS-B	run 57/100	MSE=4.939e-13	nit=16	nfev=69	t=9.72s
L-BFGS-B	run 58/100	MSE=1.716e-12	nit=13	nfev=54	t=7.61s
L-BFGS-B	run 59/100	MSE=4.054e-13	nit=12	nfev=57	t=7.95s
L-BFGS-B	run 60/100	MSE=9.698e-13	nit=19	nfev=75	t=10.64s
L-BFGS-B	run 61/100	MSE=6.780e-13	nit=11	nfev=42	t=5.91s
L-BFGS-B	run 62/100	MSE=6.388e-13	nit=7	nfev=51	t=7.19s
L-BFGS-B	run 63/100	MSE=4.615e-12	nit=12	nfev=42	t=5.76s
L-BFGS-B	run 64/100	MSE=4.982e-13	nit=12	nfev=54	t=7.55s
L-BFGS-B	run 65/100	MSE=8.007e-12	nit=10	nfev=54	t=7.57s
L-BFGS-B	run 66/100	MSE=9.455e-13	nit=13	nfev=45	t=6.38s
L-BFGS-B	run 67/100	MSE=2.338e-12	nit=10	nfev=39	t=5.52s
L-BFGS-B	run 68/100	MSE=4.968e-14	nit=9	nfev=42	t=6.01s
L-BFGS-B	run 69/100	MSE=5.237e-13	nit=15	nfev=66	t=9.31s
L-BFGS-B	run 70/100	MSE=4.058e-13	nit=15	nfev=72	t=10.17s
L-BFGS-B	run 71/100	MSE=1.435e-12	nit=11	nfev=57	t=7.96s
L-BFGS-B	run 72/100	MSE=3.455e-13	nit=19	nfev=81	t=11.31s
L-BFGS-B	run 73/100	MSE=6.405e-13	nit=13	nfev=45	t=6.37s
L-BFGS-B	run 74/100	MSE=1.459e-12	nit=12	nfev=57	t=7.99s
L-BFGS-B	run 75/100	MSE=9.893e-13	nit=12	nfev=54	t=7.63s
L-BFGS-B	run 76/100	MSE=3.239e-13	nit=14	nfev=66	t=9.19s
L-BFGS-B	run 77/100	MSE=3.720e-13	nit=13	nfev=48	t=6.73s
L-BFGS-B	run 78/100	MSE=1.208e-13	nit=9	nfev=48	t=6.72s
L-BFGS-B	run 79/100	MSE=3.563e-13	nit=14	nfev=75	t=10.42s
L-BFGS-B	run 80/100	MSE=6.139e-13	nit=16	nfev=81	t=11.55s
L-BFGS-B	run 81/100	MSE=4.466e-12	nit=14	nfev=48	t=6.77s
L-BFGS-B	run 82/100	MSE=1.262e-13	nit=9	nfev=39	t=5.50s
L-BFGS-B	run 83/100	MSE=5.208e-13	nit=13	nfev=51	t=6.96s
L-BFGS-B	run 84/100	MSE=3.139e-13	nit=10	nfev=45	t=6.11s
L-BFGS-B	run 85/100	MSE=2.969e-13	nit=11	nfev=45	t=6.37s
L-BFGS-B	run 86/100	MSE=1.321e-12	nit=18	nfev=93	t=13.10s
L-BFGS-B	run 87/100	MSE=3.301e-13	nit=10	nfev=39	t=5.55s
L-BFGS-B	run 88/100	MSE=1.961e-12	nit=11	nfev=45	t=6.24s
L-BFGS-B	run 89/100	MSE=1.262e-13	nit=12	nfev=54	t=7.64s
L-BFGS-B	run 90/100	MSE=1.525e-13	nit=19	nfev=84	t=11.83s

L-BFGS-B	run 91/100	MSE=6.318e-13	nit=13	nfev=45	t=6.35s
L-BFGS-B	run 92/100	MSE=1.253e-13	nit=9	nfev=42	t=5.89s
L-BFGS-B	run 93/100	MSE=3.027e-13	nit=10	nfev=39	t=5.37s
L-BFGS-B	run 94/100	MSE=3.752e-13	nit=12	nfev=51	t=7.24s
L-BFGS-B	run 95/100	MSE=1.002e-11	nit=11	nfev=42	t=6.04s
L-BFGS-B	run 96/100	MSE=3.386e-13	nit=13	nfev=60	t=8.43s
L-BFGS-B	run 97/100	MSE=1.700e-12	nit=14	nfev=48	t=6.78s
L-BFGS-B	run 98/100	MSE=9.466e-13	nit=13	nfev=66	t=9.20s
L-BFGS-B	run 99/100	MSE=1.234e-12	nit=13	nfev=45	t=6.19s
L-BFGS-B	run 100/100	MSE=2.612e-14	nit=9	nfev=36	t=5.09s

Scenario A - SLSQP+jac (100 runs)

SLSQP	run 1/100	MSE=3.609e-07	nit=11	nfev=36	t=5.07s
SLSQP	run 2/100	MSE=4.311e-10	nit=12	nfev=41	t=5.67s
SLSQP	run 3/100	MSE=4.679e-09	nit=10	nfev=32	t=4.52s
SLSQP	run 4/100	MSE=2.930e-10	nit=9	nfev=29	t=4.15s
SLSQP	run 5/100	MSE=9.841e-08	nit=14	nfev=48	t=6.60s
SLSQP	run 6/100	MSE=1.132e-07	nit=7	nfev=26	t=3.54s
SLSQP	run 7/100	MSE=1.433e-07	nit=11	nfev=35	t=5.00s
SLSQP	run 8/100	MSE=2.497e-07	nit=14	nfev=46	t=6.40s
SLSQP	run 9/100	MSE=4.113e-07	nit=7	nfev=23	t=3.30s
SLSQP	run 10/100	MSE=3.654e-07	nit=11	nfev=34	t=4.80s
SLSQP	run 11/100	MSE=2.782e-07	nit=5	nfev=17	t=2.38s
SLSQP	run 12/100	MSE=2.249e-08	nit=7	nfev=23	t=3.23s
SLSQP	run 13/100	MSE=1.062e-09	nit=15	nfev=49	t=6.86s
SLSQP	run 14/100	MSE=7.362e-08	nit=11	nfev=35	t=4.98s
SLSQP	run 15/100	MSE=1.546e-08	nit=8	nfev=26	t=3.58s
SLSQP	run 16/100	MSE=2.467e-09	nit=5	nfev=20	t=2.90s
SLSQP	run 17/100	MSE=5.227e-10	nit=9	nfev=30	t=4.11s
SLSQP	run 18/100	MSE=6.599e-08	nit=9	nfev=33	t=4.81s
SLSQP	run 19/100	MSE=3.126e-07	nit=11	nfev=36	t=5.12s
SLSQP	run 20/100	MSE=1.267e-08	nit=13	nfev=42	t=5.78s
SLSQP	run 21/100	MSE=1.155e-07	nit=12	nfev=38	t=5.43s
SLSQP	run 22/100	MSE=2.180e-08	nit=10	nfev=34	t=4.81s
SLSQP	run 23/100	MSE=2.839e-08	nit=10	nfev=34	t=4.63s
SLSQP	run 24/100	MSE=1.690e-09	nit=13	nfev=44	t=6.23s
SLSQP	run 25/100	MSE=1.019e-07	nit=16	nfev=54	t=7.53s
SLSQP	run 26/100	MSE=9.329e-09	nit=14	nfev=49	t=6.89s
SLSQP	run 27/100	MSE=4.648e-07	nit=15	nfev=49	t=6.86s
SLSQP	run 28/100	MSE=4.226e-07	nit=13	nfev=42	t=5.95s
SLSQP	run 29/100	MSE=5.023e-09	nit=12	nfev=39	t=5.53s
SLSQP	run 30/100	MSE=1.530e-09	nit=11	nfev=37	t=5.10s
SLSQP	run 31/100	MSE=5.247e-08	nit=10	nfev=34	t=4.84s
SLSQP	run 32/100	MSE=7.048e-08	nit=12	nfev=39	t=5.54s
SLSQP	run 33/100	MSE=7.098e-09	nit=7	nfev=24	t=3.30s
SLSQP	run 34/100	MSE=2.408e-07	nit=14	nfev=46	t=6.50s
SLSQP	run 35/100	MSE=5.031e-07	nit=16	nfev=54	t=7.54s
SLSQP	run 36/100	MSE=3.338e-08	nit=10	nfev=33	t=4.69s

SLSQP	run 37/100	MSE=3.863e-07	nit=11	nfev=35	t=4.83s
SLSQP	run 38/100	MSE=2.045e-08	nit=15	nfev=51	t=7.14s
SLSQP	run 39/100	MSE=4.858e-08	nit=11	nfev=38	t=5.18s
SLSQP	run 40/100	MSE=5.581e-08	nit=17	nfev=55	t=7.83s
SLSQP	run 41/100	MSE=9.049e-08	nit=8	nfev=27	t=3.89s
SLSQP	run 42/100	MSE=2.152e-07	nit=15	nfev=51	t=7.17s
SLSQP	run 43/100	MSE=1.192e-09	nit=7	nfev=24	t=3.28s
SLSQP	run 44/100	MSE=3.017e-08	nit=16	nfev=55	t=7.70s
SLSQP	run 45/100	MSE=1.943e-07	nit=12	nfev=41	t=5.87s
SLSQP	run 46/100	MSE=1.811e-07	nit=10	nfev=32	t=4.55s
SLSQP	run 47/100	MSE=1.550e-08	nit=15	nfev=49	t=6.87s
SLSQP	run 48/100	MSE=7.080e-10	nit=16	nfev=54	t=7.54s
SLSQP	run 49/100	MSE=1.404e-08	nit=14	nfev=48	t=6.77s
SLSQP	run 50/100	MSE=2.412e-09	nit=12	nfev=39	t=5.43s
SLSQP	run 51/100	MSE=4.119e-10	nit=13	nfev=42	t=6.02s
SLSQP	run 52/100	MSE=2.546e-08	nit=10	nfev=31	t=4.41s
SLSQP	run 53/100	MSE=1.204e-11	nit=12	nfev=38	t=5.18s
SLSQP	run 54/100	MSE=1.869e-07	nit=12	nfev=39	t=5.48s
SLSQP	run 55/100	MSE=3.708e-08	nit=15	nfev=51	t=7.10s
SLSQP	run 56/100	MSE=8.736e-10	nit=8	nfev=28	t=3.99s
SLSQP	run 57/100	MSE=3.805e-10	nit=12	nfev=41	t=5.64s
SLSQP	run 58/100	MSE=1.923e-08	nit=10	nfev=34	t=4.81s
SLSQP	run 59/100	MSE=6.667e-08	nit=12	nfev=39	t=5.53s
SLSQP	run 60/100	MSE=4.188e-09	nit=15	nfev=51	t=7.17s
SLSQP	run 61/100	MSE=3.512e-08	nit=7	nfev=23	t=3.19s
SLSQP	run 62/100	MSE=1.913e-10	nit=9	nfev=30	t=4.35s
SLSQP	run 63/100	MSE=8.078e-09	nit=15	nfev=49	t=6.88s
SLSQP	run 64/100	MSE=4.127e-08	nit=13	nfev=43	t=5.86s
SLSQP	run 65/100	MSE=2.450e-07	nit=15	nfev=52	t=7.30s
SLSQP	run 66/100	MSE=4.243e-09	nit=14	nfev=46	t=6.45s
SLSQP	run 67/100	MSE=1.469e-07	nit=12	nfev=40	t=5.68s
SLSQP	run 68/100	MSE=4.406e-09	nit=7	nfev=24	t=3.47s
SLSQP	run 69/100	MSE=2.238e-09	nit=16	nfev=52	t=7.10s
SLSQP	run 70/100	MSE=6.837e-03	nit=5	nfev=18	t=2.66s
SLSQP	run 71/100	MSE=6.587e-09	nit=13	nfev=43	t=6.09s
SLSQP	run 72/100	MSE=3.657e-07	nit=12	nfev=40	t=5.52s
SLSQP	run 73/100	MSE=5.163e-09	nit=8	nfev=27	t=3.91s
SLSQP	run 74/100	MSE=4.849e-08	nit=14	nfev=45	t=6.34s
SLSQP	run 75/100	MSE=3.269e-09	nit=8	nfev=27	t=3.70s
SLSQP	run 76/100	MSE=3.067e-08	nit=9	nfev=30	t=4.27s
SLSQP	run 77/100	MSE=2.010e-09	nit=8	nfev=27	t=3.70s
SLSQP	run 78/100	MSE=1.663e-09	nit=18	nfev=59	t=8.25s
SLSQP	run 79/100	MSE=1.331e-07	nit=10	nfev=33	t=4.69s
SLSQP	run 80/100	MSE=3.802e-07	nit=15	nfev=51	t=7.14s
SLSQP	run 81/100	MSE=6.803e-08	nit=12	nfev=43	t=6.11s
SLSQP	run 82/100	MSE=3.089e-09	nit=8	nfev=28	t=3.89s
SLSQP	run 83/100	MSE=1.217e-08	nit=11	nfev=38	t=5.38s
SLSQP	run 84/100	MSE=4.296e-10	nit=6	nfev=20	t=2.73s

SLSQP	run 85/100	MSE=4.612e-07	nit=13	nfev=42	t=6.01s
SLSQP	run 86/100	MSE=1.040e-08	nit=8	nfev=28	t=4.00s
SLSQP	run 87/100	MSE=2.484e-07	nit=9	nfev=30	t=4.09s
SLSQP	run 88/100	MSE=2.023e-09	nit=10	nfev=34	t=4.85s
SLSQP	run 89/100	MSE=3.520e-09	nit=15	nfev=50	t=7.11s
SLSQP	run 90/100	MSE=2.625e-07	nit=13	nfev=44	t=6.23s
SLSQP	run 91/100	MSE=2.535e-10	nit=9	nfev=31	t=4.31s
SLSQP	run 92/100	MSE=3.983e-07	nit=7	nfev=26	t=3.75s
SLSQP	run 93/100	MSE=6.840e-08	nit=15	nfev=54	t=7.57s
SLSQP	run 94/100	MSE=2.393e-09	nit=12	nfev=39	t=5.58s
SLSQP	run 95/100	MSE=1.206e-07	nit=8	nfev=28	t=3.86s
SLSQP	run 96/100	MSE=2.701e-07	nit=12	nfev=39	t=5.57s
SLSQP	run 97/100	MSE=5.357e-10	nit=15	nfev=50	t=7.08s
SLSQP	run 98/100	MSE=2.238e-08	nit=9	nfev=30	t=4.11s
SLSQP	run 99/100	MSE=2.842e-07	nit=13	nfev=44	t=6.20s
SLSQP	run 100/100	MSE=1.401e-07	nit=7	nfev=26	t=3.76s

Scenario A - COBYLA (100 runs)

COBYLA	run 1/100	MSE=5.728e-05	nit=-1	nfev=255	t=36.16s
COBYLA	run 2/100	MSE=6.641e-05	nit=-1	nfev=534	t=76.48s
COBYLA	run 3/100	MSE=4.989e-06	nit=-1	nfev=105	t=14.76s
COBYLA	run 4/100	MSE=7.140e-05	nit=-1	nfev=564	t=80.23s
COBYLA	run 5/100	MSE=2.544e-04	nit=-1	nfev=584	t=83.26s
COBYLA	run 6/100	MSE=5.690e-05	nit=-1	nfev=680	t=96.51s
COBYLA	run 7/100	MSE=1.228e-05	nit=-1	nfev=130	t=18.42s
COBYLA	run 8/100	MSE=1.226e-04	nit=-1	nfev=89	t=12.63s
COBYLA	run 9/100	MSE=5.892e-02	nit=-1	nfev=22	t=3.05s
COBYLA	run 10/100	MSE=4.545e-06	nit=-1	nfev=114	t=16.17s
COBYLA	run 11/100	MSE=1.481e-04	nit=-1	nfev=516	t=73.44s
COBYLA	run 12/100	MSE=5.903e-02	nit=-1	nfev=23	t=3.37s
COBYLA	run 13/100	MSE=1.155e-05	nit=-1	nfev=72	t=10.19s
COBYLA	run 14/100	MSE=2.679e-06	nit=-1	nfev=102	t=14.51s
COBYLA	run 15/100	MSE=6.825e-03	nit=-1	nfev=32	t=4.43s
COBYLA	run 16/100	MSE=1.061e-05	nit=-1	nfev=109	t=15.71s
COBYLA	run 17/100	MSE=1.434e-04	nit=-1	nfev=567	t=80.64s
COBYLA	run 18/100	MSE=8.973e-06	nit=-1	nfev=91	t=12.95s
COBYLA	run 19/100	MSE=1.719e-05	nit=-1	nfev=572	t=81.20s
COBYLA	run 20/100	MSE=1.601e-05	nit=-1	nfev=104	t=14.80s
COBYLA	run 21/100	MSE=6.151e-05	nit=-1	nfev=230	t=32.78s
COBYLA	run 22/100	MSE=6.549e-05	nit=-1	nfev=67	t=9.50s
COBYLA	run 23/100	MSE=1.010e-04	nit=-1	nfev=98	t=14.20s
COBYLA	run 24/100	MSE=2.460e-05	nit=-1	nfev=85	t=12.22s
COBYLA	run 25/100	MSE=1.121e-04	nit=-1	nfev=657	t=96.90s
COBYLA	run 26/100	MSE=1.852e-06	nit=-1	nfev=148	t=24.04s
COBYLA	run 27/100	MSE=1.499e-05	nit=-1	nfev=83	t=13.25s
COBYLA	run 28/100	MSE=3.672e-05	nit=-1	nfev=88	t=16.43s
COBYLA	run 29/100	MSE=4.466e-05	nit=-1	nfev=532	t=90.83s
COBYLA	run 30/100	MSE=1.795e-05	nit=-1	nfev=119	t=20.34s

COBYLA	run 31/100	MSE=6.109e-05	nit=-1	nfev=121	t=24.17s
COBYLA	run 32/100	MSE=3.368e-06	nit=-1	nfev=152	t=25.93s
COBYLA	run 33/100	MSE=6.683e-03	nit=-1	nfev=51	t=10.50s
COBYLA	run 34/100	MSE=1.359e-05	nit=-1	nfev=116	t=19.87s
COBYLA	run 35/100	MSE=3.267e-05	nit=-1	nfev=87	t=14.70s
COBYLA	run 36/100	MSE=3.205e-07	nit=-1	nfev=73	t=12.22s
COBYLA	run 37/100	MSE=1.386e-04	nit=-1	nfev=574	t=89.38s
COBYLA	run 38/100	MSE=3.092e-05	nit=-1	nfev=145	t=22.75s
COBYLA	run 39/100	MSE=2.858e-05	nit=-1	nfev=576	t=89.39s
COBYLA	run 40/100	MSE=1.173e-07	nit=-1	nfev=67	t=10.26s
COBYLA	run 41/100	MSE=7.949e-05	nit=-1	nfev=50	t=7.79s
COBYLA	run 42/100	MSE=5.900e-02	nit=-1	nfev=31	t=4.97s
COBYLA	run 43/100	MSE=1.972e-05	nit=-1	nfev=548	t=85.83s
COBYLA	run 44/100	MSE=5.903e-02	nit=-1	nfev=23	t=3.50s
COBYLA	run 45/100	MSE=2.394e-05	nit=-1	nfev=133	t=24.67s
COBYLA	run 46/100	MSE=1.677e-05	nit=-1	nfev=153	t=24.92s
COBYLA	run 47/100	MSE=1.134e-05	nit=-1	nfev=97	t=15.97s
COBYLA	run 48/100	MSE=1.001e-04	nit=-1	nfev=256	t=41.87s
COBYLA	run 49/100	MSE=1.719e-05	nit=-1	nfev=571	t=95.60s
COBYLA	run 50/100	MSE=7.285e-05	nit=-1	nfev=84	t=14.34s
COBYLA	run 51/100	MSE=1.561e-04	nit=-1	nfev=116	t=19.65s
COBYLA	run 52/100	MSE=4.146e-05	nit=-1	nfev=52	t=8.75s
COBYLA	run 53/100	MSE=5.859e-05	nit=-1	nfev=82	t=13.57s
COBYLA	run 54/100	MSE=2.900e-05	nit=-1	nfev=109	t=18.10s
COBYLA	run 55/100	MSE=2.102e-05	nit=-1	nfev=80	t=13.33s
COBYLA	run 56/100	MSE=1.178e-04	nit=-1	nfev=582	t=98.18s
COBYLA	run 57/100	MSE=1.517e-07	nit=-1	nfev=82	t=14.45s
COBYLA	run 58/100	MSE=1.386e-04	nit=-1	nfev=574	t=100.83s
COBYLA	run 59/100	MSE=4.634e-05	nit=-1	nfev=41	t=8.28s
COBYLA	run 60/100	MSE=5.532e-06	nit=-1	nfev=156	t=26.24s
COBYLA	run 61/100	MSE=8.683e-05	nit=-1	nfev=639	t=108.92s
COBYLA	run 62/100	MSE=9.755e-05	nit=-1	nfev=99	t=16.81s
COBYLA	run 63/100	MSE=1.565e-06	nit=-1	nfev=143	t=30.31s
COBYLA	run 64/100	MSE=5.112e-05	nit=-1	nfev=109	t=17.31s
COBYLA	run 65/100	MSE=4.137e-05	nit=-1	nfev=27	t=4.12s
COBYLA	run 66/100	MSE=2.710e-06	nit=-1	nfev=101	t=17.22s
COBYLA	run 67/100	MSE=1.719e-05	nit=-1	nfev=571	t=91.54s
COBYLA	run 68/100	MSE=2.622e-05	nit=-1	nfev=548	t=85.03s
COBYLA	run 69/100	MSE=1.719e-05	nit=-1	nfev=572	t=89.61s
COBYLA	run 70/100	MSE=1.782e-04	nit=-1	nfev=61	t=9.56s
COBYLA	run 71/100	MSE=1.669e-05	nit=-1	nfev=85	t=12.99s
COBYLA	run 72/100	MSE=4.974e-05	nit=-1	nfev=102	t=16.15s
COBYLA	run 73/100	MSE=5.900e-02	nit=-1	nfev=30	t=4.52s
COBYLA	run 74/100	MSE=1.473e-05	nit=-1	nfev=120	t=18.68s
COBYLA	run 75/100	MSE=6.848e-05	nit=-1	nfev=160	t=24.76s
COBYLA	run 76/100	MSE=3.139e-05	nit=-1	nfev=103	t=16.77s
COBYLA	run 77/100	MSE=1.175e-05	nit=-1	nfev=53	t=8.16s
COBYLA	run 78/100	MSE=5.270e-05	nit=-1	nfev=643	t=98.98s

COBYLA	run 79/100	MSE=1.422e-05	nit=-1	nfev=108	t=16.82s
COBYLA	run 80/100	MSE=3.860e-05	nit=-1	nfev=119	t=18.30s
COBYLA	run 81/100	MSE=6.772e-07	nit=-1	nfev=61	t=9.76s
COBYLA	run 82/100	MSE=4.061e-05	nit=-1	nfev=641	t=99.91s
COBYLA	run 83/100	MSE=1.381e-05	nit=-1	nfev=123	t=18.99s
COBYLA	run 84/100	MSE=7.211e-05	nit=-1	nfev=233	t=38.58s
COBYLA	run 85/100	MSE=5.903e-02	nit=-1	nfev=20	t=3.31s
COBYLA	run 86/100	MSE=5.885e-02	nit=-1	nfev=40	t=8.58s
COBYLA	run 87/100	MSE=4.122e-05	nit=-1	nfev=113	t=17.77s
COBYLA	run 88/100	MSE=5.903e-02	nit=-1	nfev=23	t=3.52s
COBYLA	run 89/100	MSE=3.235e-05	nit=-1	nfev=93	t=14.39s
COBYLA	run 90/100	MSE=2.028e-05	nit=-1	nfev=79	t=12.25s
COBYLA	run 91/100	MSE=5.903e-02	nit=-1	nfev=23	t=3.36s
COBYLA	run 92/100	MSE=7.958e-05	nit=-1	nfev=502	t=77.77s
COBYLA	run 93/100	MSE=1.383e-06	nit=-1	nfev=115	t=17.58s
COBYLA	run 94/100	MSE=5.402e-05	nit=-1	nfev=114	t=17.92s
COBYLA	run 95/100	MSE=3.380e-07	nit=-1	nfev=131	t=20.38s
COBYLA	run 96/100	MSE=2.540e-06	nit=-1	nfev=81	t=12.74s
COBYLA	run 97/100	MSE=2.438e-05	nit=-1	nfev=119	t=18.16s
COBYLA	run 98/100	MSE=5.903e-02	nit=-1	nfev=24	t=3.53s
COBYLA	run 99/100	MSE=8.058e-06	nit=-1	nfev=143	t=22.05s
COBYLA	run 100/100	MSE=6.500e-05	nit=-1	nfev=662	

t=105.12s

Scenario A - trust-constr (100 runs)

trust-constr	run 1/100	MSE=4.216e-13	nit=29	nfev=63	t=9.69s
trust-constr	run 2/100	MSE=8.766e-13	nit=27	nfev=63	t=9.65s
trust-constr	run 3/100	MSE=8.766e-13	nit=40	nfev=105	t=16.34s
trust-constr	run 4/100	MSE=3.743e-14	nit=71	nfev=189	t=29.00s
trust-constr	run 5/100	MSE=7.575e-14	nit=21	nfev=42	t=6.55s
trust-constr	run 6/100	MSE=7.571e-14	nit=42	nfev=105	t=18.14s
trust-constr	run 7/100	MSE=7.572e-14	nit=31	nfev=78	t=13.56s
trust-constr	run 8/100	MSE=7.571e-14	nit=28	nfev=63	t=11.03s
trust-constr	run 9/100	MSE=2.714e-12	nit=25	nfev=54	t=9.24s
trust-constr	run 10/100	MSE=7.572e-14	nit=46	nfev=120	t=18.91s
trust-constr	run 11/100	MSE=2.714e-12	nit=28	nfev=63	t=10.15s
trust-constr	run 12/100	MSE=2.714e-12	nit=26	nfev=57	t=9.46s
trust-constr	run 13/100	MSE=7.573e-14	nit=27	nfev=60	t=10.01s
trust-constr	run 14/100	MSE=7.570e-14	nit=22	nfev=45	t=7.14s
trust-constr	run 15/100	MSE=7.591e-14	nit=22	nfev=45	t=7.51s
trust-constr	run 16/100	MSE=5.634e-12	nit=20	nfev=42	t=7.48s
trust-constr	run 17/100	MSE=4.983e-15	nit=43	nfev=117	t=18.96s
trust-constr	run 18/100	MSE=8.766e-13	nit=52	nfev=138	t=23.14s
trust-constr	run 19/100	MSE=3.504e-13	nit=31	nfev=69	t=11.19s
trust-constr	run 20/100	MSE=5.635e-12	nit=23	nfev=51	t=8.63s
trust-constr	run 21/100	MSE=7.565e-14	nit=37	nfev=93	t=15.48s
trust-constr	run 22/100	MSE=7.580e-14	nit=32	nfev=81	t=13.03s
trust-constr	run 23/100	MSE=8.766e-13	nit=31	nfev=75	t=12.16s

trust-constr	run 24/100	MSE=8.766e-13	nit=47	nfev=123	t=19.99s
trust-constr	run 25/100	MSE=5.170e-15	nit=27	nfev=63	t=10.29s
trust-constr	run 26/100	MSE=5.633e-12	nit=23	nfev=51	t=8.79s
trust-constr	run 27/100	MSE=5.591e-15	nit=86	nfev=234	t=38.71s
trust-constr	run 28/100	MSE=8.766e-13	nit=61	nfev=165	t=26.94s
trust-constr	run 29/100	MSE=2.423e-16	nit=81	nfev=222	t=38.51s
trust-constr	run 30/100	MSE=7.573e-14	nit=32	nfev=75	t=13.26s
trust-constr	run 31/100	MSE=4.215e-13	nit=29	nfev=63	t=11.17s
trust-constr	run 32/100	MSE=8.766e-13	nit=27	nfev=60	t=11.63s
trust-constr	run 33/100	MSE=5.633e-12	nit=34	nfev=90	t=16.16s
trust-constr	run 34/100	MSE=9.636e-16	nit=91	nfev=249	t=44.73s
trust-constr	run 35/100	MSE=1.120e-15	nit=78	nfev=210	t=35.60s
trust-constr	run 36/100	MSE=6.705e-14	nit=58	nfev=147	t=25.86s
trust-constr	run 37/100	MSE=8.888e-14	nit=30	nfev=66	t=10.69s
trust-constr	run 38/100	MSE=7.577e-14	nit=26	nfev=57	t=9.16s
trust-constr	run 39/100	MSE=7.381e-14	nit=64	nfev=165	t=25.71s
trust-constr	run 40/100	MSE=9.626e-15	nit=87	nfev=237	t=42.42s
trust-constr	run 41/100	MSE=4.131e-15	nit=72	nfev=192	t=30.12s
trust-constr	run 42/100	MSE=5.036e-15	nit=24	nfev=57	t=9.53s
trust-constr	run 43/100	MSE=2.714e-12	nit=28	nfev=63	t=10.43s
trust-constr	run 44/100	MSE=2.714e-12	nit=255	nfev=747	t=118.20s
trust-constr	run 45/100	MSE=8.765e-13	nit=26	nfev=57	t=9.24s
trust-constr	run 46/100	MSE=5.637e-12	nit=23	nfev=51	t=8.49s
trust-constr	run 47/100	MSE=4.988e-15	nit=26	nfev=60	t=10.03s
trust-constr	run 48/100	MSE=8.766e-13	nit=25	nfev=54	t=8.20s
trust-constr	run 49/100	MSE=4.977e-15	nit=37	nfev=81	t=12.83s
trust-constr	run 50/100	MSE=7.573e-14	nit=59	nfev=150	t=24.77s
trust-constr	run 51/100	MSE=7.573e-14	nit=21	nfev=42	t=6.80s
trust-constr	run 52/100	MSE=2.192e-16	nit=140	nfev=402	t=61.21s
trust-constr	run 53/100	MSE=4.111e-16	nit=96	nfev=264	t=38.32s
trust-constr	run 54/100	MSE=7.568e-14	nit=46	nfev=117	t=16.92s
trust-constr	run 55/100	MSE=7.558e-14	nit=27	nfev=63	t=9.22s
trust-constr	run 56/100	MSE=2.714e-12	nit=22	nfev=45	t=6.54s
trust-constr	run 57/100	MSE=5.635e-12	nit=22	nfev=48	t=7.01s
trust-constr	run 58/100	MSE=8.765e-13	nit=31	nfev=75	t=11.05s
trust-constr	run 59/100	MSE=5.329e-15	nit=26	nfev=63	t=9.26s
trust-constr	run 60/100	MSE=5.029e-15	nit=22	nfev=48	t=6.99s
trust-constr	run 61/100	MSE=2.713e-12	nit=29	nfev=66	t=9.48s
trust-constr	run 62/100	MSE=6.449e-16	nit=93	nfev=264	t=38.23s
trust-constr	run 63/100	MSE=5.637e-12	nit=22	nfev=48	t=6.98s
trust-constr	run 64/100	MSE=7.580e-14	nit=38	nfev=96	t=13.81s
trust-constr	run 65/100	MSE=1.241e-16	nit=79	nfev=213	t=31.20s
trust-constr	run 66/100	MSE=5.633e-12	nit=21	nfev=45	t=6.58s
trust-constr	run 67/100	MSE=7.564e-14	nit=43	nfev=108	t=16.02s
trust-constr	run 68/100	MSE=2.714e-12	nit=38	nfev=96	t=14.03s
trust-constr	run 69/100	MSE=7.597e-14	nit=65	nfev=168	t=24.18s
trust-constr	run 70/100	MSE=7.573e-14	nit=25	nfev=57	t=8.27s
trust-constr	run 71/100	MSE=5.633e-12	nit=30	nfev=72	t=10.48s

trust-constr	run 72/100	MSE=6.976e-14	nit=68	nfev=180	t=26.11s
trust-constr	run 73/100	MSE=2.714e-12	nit=29	nfev=66	t=9.53s
trust-constr	run 74/100	MSE=2.369e-15	nit=115	nfev=324	t=46.91s
trust-constr	run 75/100	MSE=8.001e-18	nit=78	nfev=210	t=30.18s
trust-constr	run 76/100	MSE=5.633e-12	nit=23	nfev=51	t=7.46s
trust-constr	run 77/100	MSE=7.545e-14	nit=21	nfev=42	t=6.13s
trust-constr	run 78/100	MSE=4.952e-15	nit=19	nfev=39	t=5.56s
trust-constr	run 79/100	MSE=5.633e-12	nit=29	nfev=69	t=10.07s
trust-constr	run 80/100	MSE=2.318e-12	nit=20	nfev=45	t=6.53s
trust-constr	run 81/100	MSE=5.636e-12	nit=22	nfev=48	t=6.77s
trust-constr	run 82/100	MSE=6.358e-16	nit=94	nfev=258	t=37.28s
trust-constr	run 83/100	MSE=5.633e-12	nit=23	nfev=51	t=7.38s
trust-constr	run 84/100	MSE=8.766e-13	nit=31	nfev=72	t=10.53s
trust-constr	run 85/100	MSE=7.532e-14	nit=58	nfev=147	t=21.01s
trust-constr	run 86/100	MSE=7.569e-14	nit=35	nfev=87	t=12.64s
trust-constr	run 87/100	MSE=8.889e-14	nit=28	nfev=60	t=8.71s
trust-constr	run 88/100	MSE=1.301e-10	nit=25	nfev=57	t=8.25s
trust-constr	run 89/100	MSE=7.559e-14	nit=26	nfev=57	t=8.31s
trust-constr	run 90/100	MSE=8.765e-13	nit=23	nfev=48	t=7.06s
trust-constr	run 91/100	MSE=2.714e-12	nit=31	nfev=72	t=10.39s
trust-constr	run 92/100	MSE=2.461e-14	nit=106	nfev=297	t=45.76s
trust-constr	run 93/100	MSE=8.765e-13	nit=23	nfev=48	t=7.75s
trust-constr	run 94/100	MSE=4.929e-15	nit=86	nfev=249	t=37.84s
trust-constr	run 95/100	MSE=5.077e-15	nit=22	nfev=51	t=7.40s
trust-constr	run 96/100	MSE=8.768e-13	nit=28	nfev=66	t=10.29s
trust-constr	run 97/100	MSE=5.635e-12	nit=23	nfev=51	t=7.93s
trust-constr	run 98/100	MSE=8.766e-13	nit=28	nfev=63	t=9.13s
trust-constr	run 99/100	MSE=4.217e-13	nit=24	nfev=48	t=7.38s
trust-constr	run 100/100	MSE=7.572e-14	nit=21	nfev=42	t=5.88s

=====

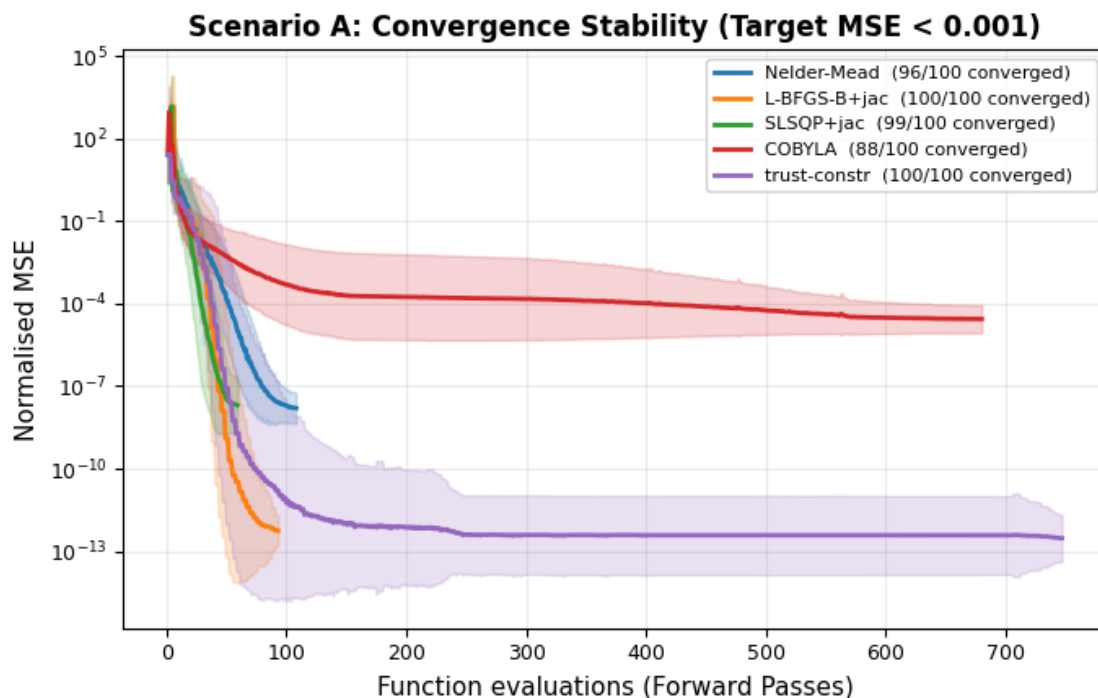
SCENARIO A: Robustness (Tol < 0.001) & Discovered Physics State vs Reference

=====

method_tag	conv_rate	nit_mean	nfev_mean	wall_time_mean	final_mse_mean	I_1_ref
I_1	I_3_ref	I_3	alpha_x	alpha_y		
COBYLA	88%	208.9	208.9	32.716 s	2.18e-05	0.8218 A
0.7511 A	1.0430 A	0.5806 A	1.15e-03	2.34e-03		
L-BFGS-B+jac	100%	12.2	53.8	7.558 s	5.68e-13	0.8218 A
0.7512 A	1.0430 A	0.5806 A	2.43e-07	3.89e-07		
Nelder-Mead	96%	39.4	75.3	11.091 s	1.12e-08	0.8218 A
0.2502 A	1.0430 A	0.2403 A	3.27e-05	1.49e-05		
SLSQP+jac	99%	11.3	37.9	5.323 s	2.04e-08	0.8218 A
0.2502 A	1.0430 A	0.2403 A	4.69e-06	-1.45e-06		
trust-constr	100%	43.0	109.0	17.066 s	1.38e-13	0.8218 A
0.7512 A	1.0430 A	0.5806 A	5.99e-08	2.04e-07		

=====

=====



Scenario A: Best Optimised Quadrupole Currents per Method

Method	Best MSE	I (A)	I2 (A)
Nelder-Mead	6.4634e-10	0.2502	0.2403
L-BFGS-B+jac	2.6120e-14	0.7512	0.5806
SLSQP+jac	1.2036e-11	0.2502	0.2403
COBYLA	1.1734e-07	0.7511	0.5806
trust-constr	8.0014e-18	0.7512	0.5806

Reference FELsim S1 currents: I=0.8218 A, I2=1.0430 A

Overall best across all methods/runs:

Method: trust-constr, MSE=8.0014e-18

I=0.75118 A, I2=0.58057 A

time: 2h 3min 34s (started: 2026-06-05 15:42:17 -10:00)

```
[ ]: # save the results
```